

Integrated Nutrition & Supplement Management System (NutriFusion)

Term 8 Technical Documentation

Project	Nutrifusion – Integrated Nutrition & Supplement Management System
Institution	High Performance Sports Institute (HPSI)
Last Updated	April 2026
Status	Production — All Core Modules Complete

Contents

1. Project Overview	5
1.1 Background	5
1.2 Objectives.....	5
1.3 Scope	5
2. System Architecture	6
2.1 Overview	6
2.2 Communication Patterns	7
Frontend → Backend	7
Backend → Python Services.....	7
Backend → Database	7
2.3 Architectural Patterns	8
Layered Architecture (MVC-inspired)	8
Module Pattern	8
Two-Tier Validation	8
3. Technology Stack	9
3.1 Frontend.....	9
3.2 Backend	9
3.3 Database.....	9
3.4 Python Services.....	10
4. Project Structure	11
5. Module Documentation	11
5.1 SSS — Supplement Support System.....	12
Supplement Statuses	12
Supplement Library.....	12
Inventory Management	13
OCR Pipeline	13
Webscraper Pipeline.....	15
Batch Testing Pipeline	18
Supplement Similarity Search	20
5.2 AMS — Athlete Management System.....	22
Dashboard	22
Athlete Profile Management.....	23
5.3 Consultation Module	23
Consultation Session Management.....	23
The Nine-Card Framework.....	24

5.4 Admin Module	24
5.5 Auth Module	25
5.6 OCR Services.....	27
6. Database Design	28
6.1 Schema Overview	28
6.2 Database Functions.....	30
6.3 Key Design Decisions.....	31
Normalisation to Third Normal Form	31
UUIDv7 Primary Keys	31
Dynamic Lookup Tables.....	31
JSONB and Array Column Types.....	32
Inventory Ticketing System	32
No ORM (Object-Relational Mapping).....	32
6.4 Security, Compliance & Automation.....	32
Immutable Audit Trail	32
Cascading State Synchronisation	32
6.5 Database-Level Computation	33
6.6.....	33
7. API Reference.....	35
7.1 SSS Endpoints	35
Inventory	35
Staging	35
Catalog	36
Scraping	37
Supplements.....	37
Lookups	38
7.2 AMS Endpoints.....	39
Athlete	39
Coaches	40
Nutritionists	41
Sports	43
7.3 Auth Endpoints	43
7.4 Admin Endpoints	44
7.5 OCR/Python Endpoints.....	45
7.6 Consultation Endpoints.....	47
Actionables (Open Items).....	47

Anthropometry	47
Lookups	48
Session	48
Meal Log	49
Medical History	49
Nutrition Diagnosis Summary	49
Nutrition Requirements	50
Supplement Dispensing	50
Training Schedule	51
8. Security Architecture	53
8.1 Implemented Controls	53
8.2 Planned Controls	53
8.3 Authentication Flow	53
9. Setup & Deployment System setup and environment	55
Prerequisites	55
Environment Configuration	55
Starting the System	56
Verifying the Installation	57
Smoke Test	57
Common Management Commands	57
Optional: Local Database	57
Troubleshooting	58
Email Setup	58
10. Design Decisions & Patterns	59
10.1 Why Separate Python Service?	59
10.2 Why PostgreSQL with pgvector over a Dedicated Vector DB?	59
10.3 Why Parameterised Queries over an ORM?	59
10.4 Why Zod over Joi?	59
10.5 Common Code Patterns	59
Dynamic PATCH (field mapping)	59
Bulk Delete	59
Fire-and-forget Scraping	59
Transaction Pattern for Multi-table Creates	60

1. Project Overview

1.1 Background

The High Performance Sports Institute (HPSI) required a centralised platform to manage the supplement lifecycle for its athletes. Previously, nutritionists tracked supplement inventory, athlete consultations, and batch certification compliance through disconnected tools and manual processes. NutriFusion consolidates these workflows into a single, role-based web application.

1.2 Objectives

- Provide a searchable supplement library with inventory and batch tracking
- Enable structured nutrition consultations linked to athlete profiles
- Automate supplement discovery via web scraping and AI-assisted OCR label scanning
- Verify supplement batch certification against six international databases
- Enforce role-based access control for nutritionists and administrators

1.3 Scope

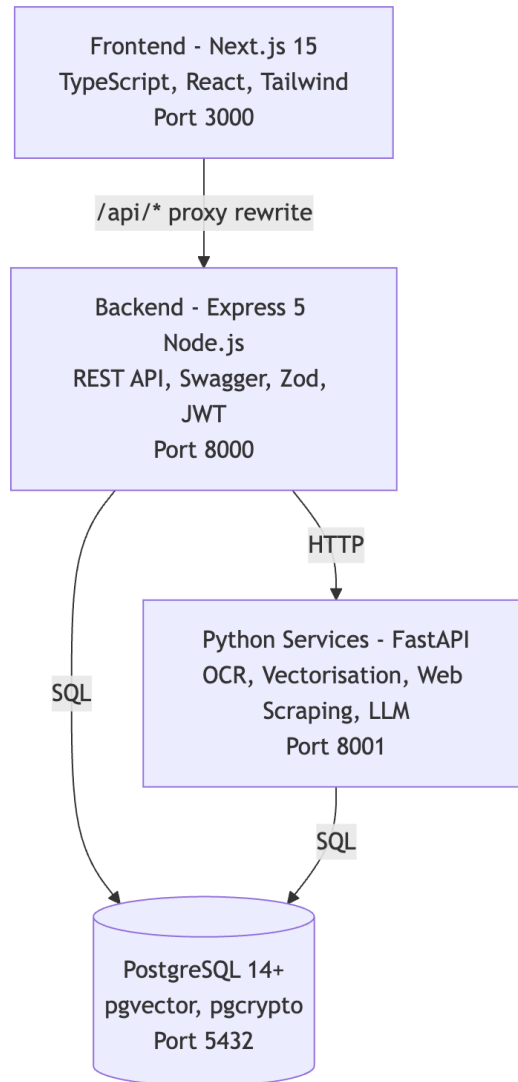
The system covers the following functional areas:

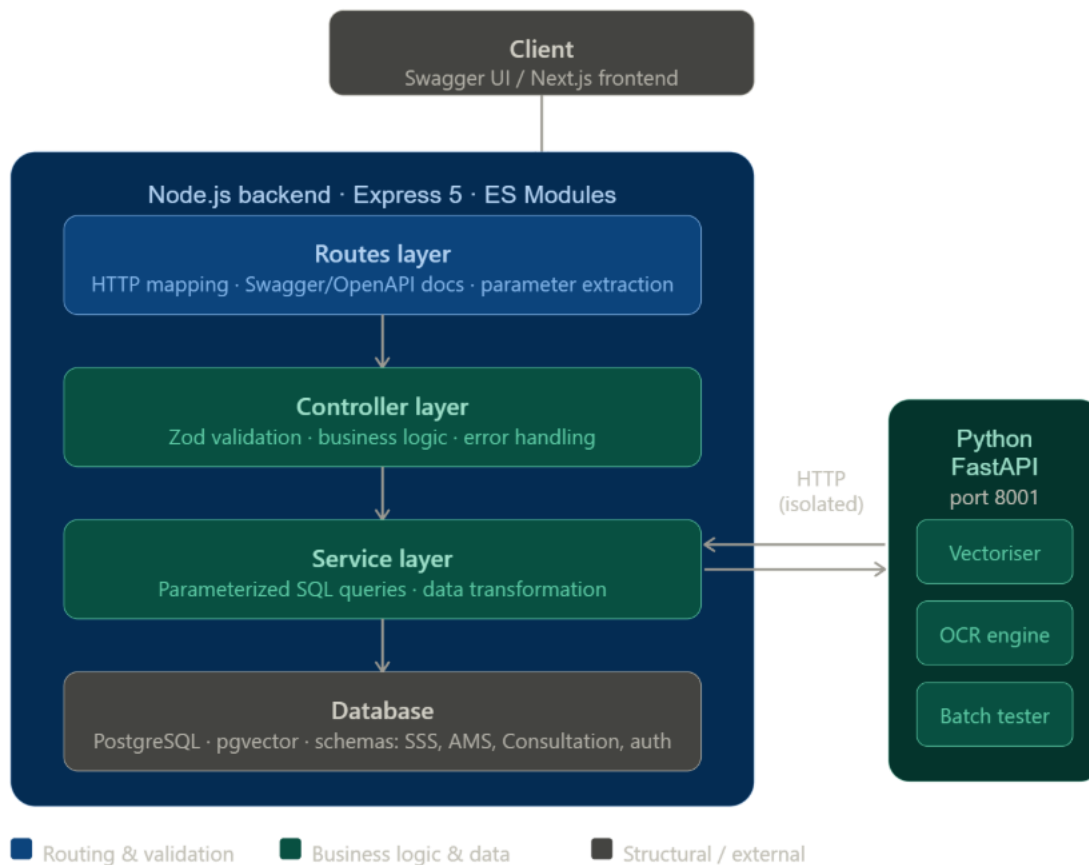
Module	Description
SSS	Supplement library, inventory batches, web scraper staging, vector similarity search
AMS	Athlete profiles, registry, medical records, consultation session linking
Consultation	Card-based clinical workflow — 8 data cards per session
Admin	User account management, sports and coach records, audit logging
Auth	Email-based two-factor authentication with JWT
OCR Services	Supplement label scanning, batch certification verification

2. System Architecture

2.1 Overview

NutriFusion is a **three-tier web application** composed of three independently running services:





2.2 Communication Patterns

Frontend → Backend

The Next.js frontend calls relative API paths (e.g. `/api/SSS/supplements`). A rewrite rule in `next.config.ts` transparently proxies these requests to `http://localhost:8000/api/*`, so no CORS handling is needed from the browser.

Backend → Python Services

The Express backend delegates AI/ML tasks to the Python FastAPI microservice via HTTP. These calls are made for:

- OCR text extraction and LLM-based structuring
- Supplement embedding vector generation
- Web scraper job execution
- Certification database verification (via Playwright)

Backend → Database

All database access uses the `pg` (node-postgres) library with a connection pool and parameterised SQL queries. No ORM is used.

2.3 Architectural Patterns

Layered Architecture (MVC-inspired)

Each backend module is structured across four layers:

Layer	File	Responsibility
Routes	routes.js	HTTP method + path routing, Swagger JSDoc annotations
Controller	controller.js	Input validation (Zod), business logic, response formatting
Service	services.js	Parameterised SQL queries, data transformation
Validation	validation.js	Reusable Zod schemas and business rule validators

Module Pattern

Every feature is self-contained in its own directory under `Backend/modules/`. Complex modules (SSS, AMS, Consultation) further split into sub-modules, each with its own routes/controller/services/validation.

Two-Tier Validation

```
Tier 1 (Zod schema) - enforces data types, required fields, formats
Tier 2 (Controller) - enforces business rules (e.g. batch_testing_org required when
status = BATCH TESTED)
```


3. Technology Stack

3.1 Frontend

Technology	Version	Purpose
Next.js	15 (App Router)	React framework, file-based routing, API proxying
React	19	UI component library
TypeScript	5.x	Static typing
Tailwind CSS	3.x	Utility-first styling
JWT (client-side)	—	Authentication state via AuthContext

3.2 Backend

Technology	Version	Purpose
Node.js	18+	JavaScript runtime
Express	5.x	HTTP server and routing
Zod	3.x	Schema validation
jsonwebtoken	—	JWT generation and verification
bcrypt	—	Password hashing
Nodemailer	—	2FA email delivery
express-rate-limit	—	Brute-force protection on auth endpoints
Swagger / OpenAPI	3.0	Interactive API documentation
pg (node-postgres)	—	PostgreSQL client with connection pooling

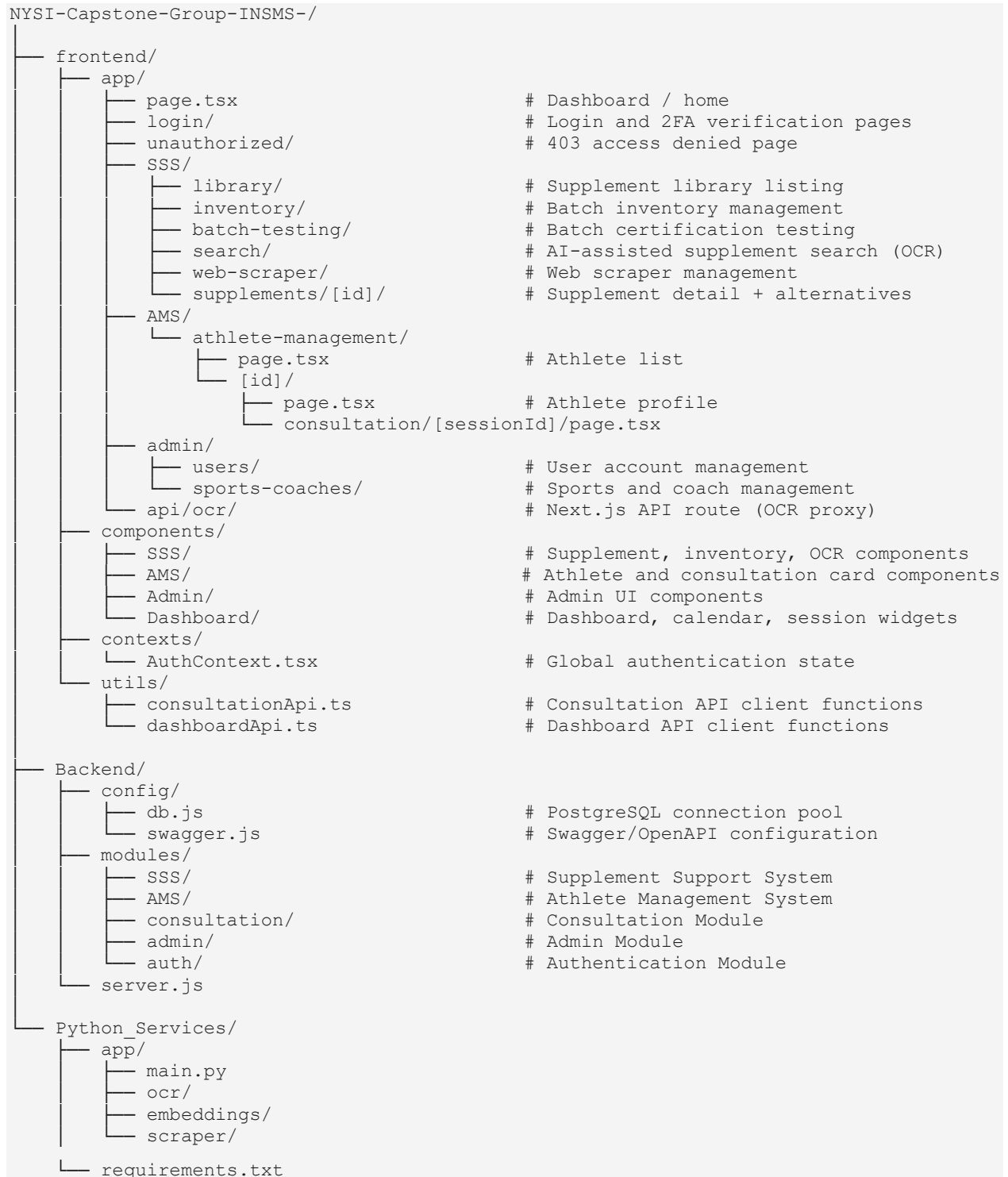
3.3 Database

Technology	Version	Purpose
PostgreSQL	14+	Relational database
pgvector	—	384-dim vector storage and cosine similarity search
pgcrypto	—	gen_random_uuid() for UUID primary keys

3.4 Python Services

Technology	Version	Purpose
Python	3.10+	Runtime
FastAPI	0.109.0	HTTP microservice framework
PaddleOCR	2.8.1	Supplement label text extraction
PaddlePaddle	2.6.2	Deep learning backend for PaddleOCR
OpenAI GPT-4o-mini	—	Structured data extraction from raw OCR text
sentence-transformers	—	BAAI/bge-small-en-v1.5 (384-dim) embedding generation
PyTorch	2.0.1 (CPU)	Model inference backend
Playwright	1.57.0	Headless browser for scraping and certification verification
ScrapeGraphAI	1.62.0	AI processing pipeline for webscraped HTML
Selenium	4.40.0	Headless browser for scraping and certification verification

4. Project Structure



5. Module Documentation

All the sections mentioned are referenced from our final report.

5.1 SSS — Supplement Support System

The SSS manages the core supplement data lifecycle, from data ingestion through web scraping to inventory management and AI-powered search.

Sub-module	Key Functionality
Supplement Library	CRUD for supplement records; links to inventory and batches
Inventory	Batch lot management: quantity, expiry, supplier, status
Batch Testing	Certification verification across 6 international databases via Playwright
Web Scraper	Trigger scrape jobs; review and approve/reject staged results
OCR / Search	Image upload → PaddleOCR → GPT-4o-mini → vector search for alternatives
Similarity Search	Generate and store 384-dim vectors for all supplement records

Supplement Statuses

Status	Meaning
ACTIVE	Available for use
INACTIVE	Not currently in use
BATCH TESTED	Certified batch on record (requires batch_testing_org)
DISCONTINUED	No longer available

Supplement Library

The Supplement Library presents the full catalogue of supplements stored in `sss.supplement`, supporting multi-field search across five attributes: *supplement name*, *supplement brand*, *ingredient list*, *packaging form*, and *supplement status*. Search is case-insensitive and uses partial matching, so a query term does not need to match the full field value. For multi-word queries, an AND logic is applied — each word in the query must appear in at least one of the searchable fields for a record to be returned, ensuring that results are progressively narrowed rather than broadened as queries become more specific. Packaging form and supplement status are resolved via joins to their respective lookup tables (`sss.supplement_packaging_form_lookup` and `sss.supplement_status_lookup`), and ingredient search is performed by casting the JSONB array in `supplement_ingredient` to text before matching. Results are returned in a relevance-ranked order that prioritises matches on supplement name, followed by brand, ingredient, packaging form, and status, reflecting the likelihood that a name or brand match is more operationally specific than a packaging or status match.

Each supplement in the library links through to a detail view that presents the supplement's full profile — ingredient list, nutritional information per 100g and per serving, certifications, warnings, and additional metadata — alongside a complete listing of all inventory batches associated with that supplement. Each batch is displayed individually with its own stock figures, allowing nutritionists to assess the availability and condition of each lot separately rather than relying on an aggregated stock summary that would obscure batch-level differences such as varying expiration dates or prices.

Inventory Management

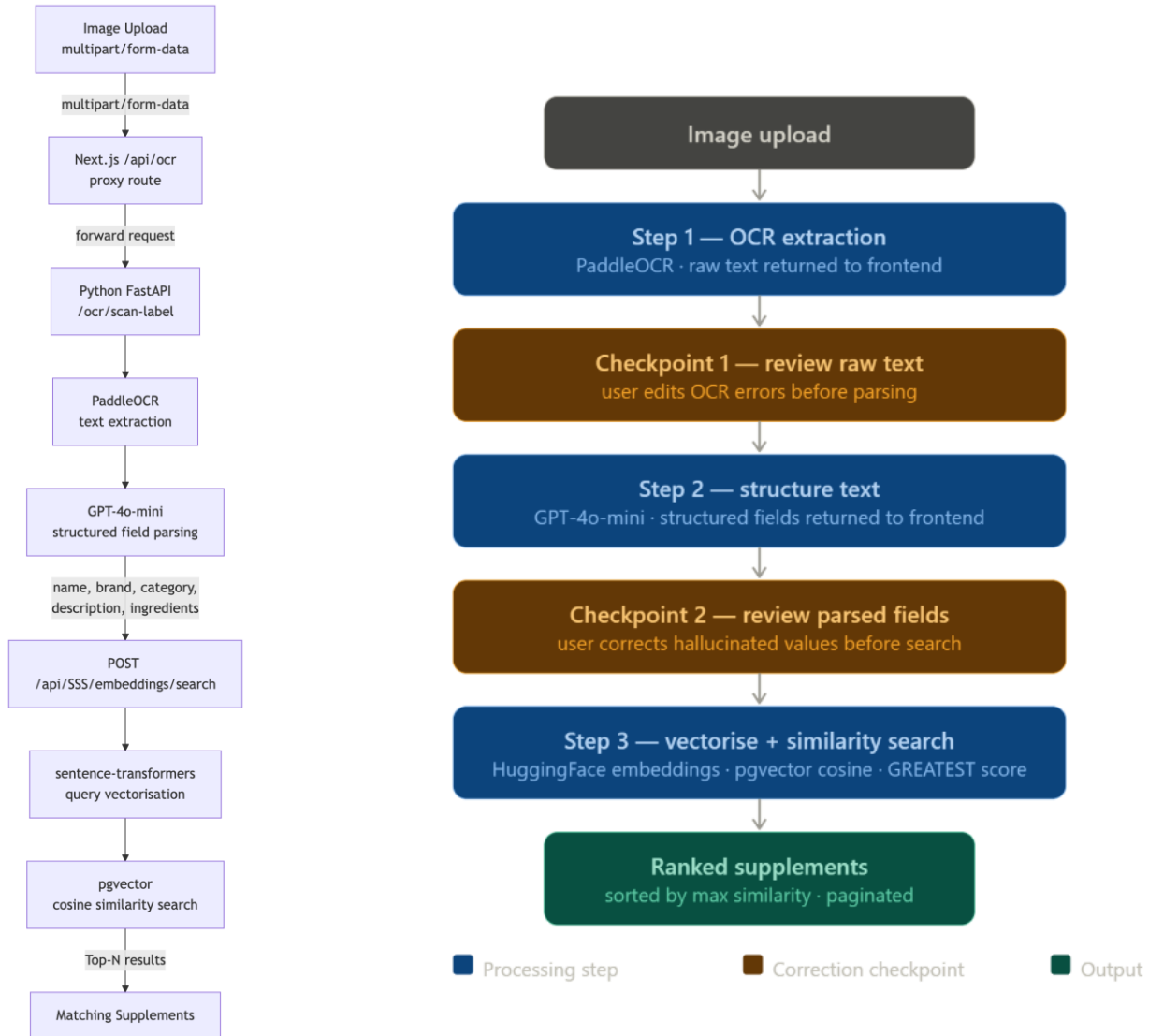
The Inventory view presents the same batch-level data from a different perspective, supporting search across batch number, supplement name, supplement brand, and batch stock status. Multi-word AND logic applies here as well. Each batch record on `sss.inventory_batch` carries the batch number, initial quantity, price, unit of measurement (defined by the user at batch creation), manufacture date, expiration date, and a stock status that reflects the current availability of that batch.

Stock availability is not stored as a static field. Instead, it is calculated dynamically each time an inventory ticket is created, by summing the quantities of all active `sss.inventory_ticket` records associated with a given batch and subtracting that total from `batch_initial_quantity`. This available quantity is then used to update the batch's stock status according to the following rules: if the ratio of available quantity to initial quantity is zero, the status is set to OUT OF STOCK; if the ratio is greater than zero but less than or equal to 0.2 (20%), the status is set to LOW STOCK; otherwise the status is set to AVAILABLE. These thresholds were determined in consultation with HPSI's nutrition team to reflect operationally meaningful stock levels that support timely reordering decisions.

The inventory ticketing system that drives this calculation records every supplement dispensing event as a row in `sss.inventory_ticket`, linking the dispensed batch (`inventory_batch_id`), the recipient athlete (`athlete_id`), and the associated consultation prescription (`prescription_id`), with a quantity and a ticket status. Tickets are created at the point of dispensing within the Supplement Dispensing card of a consultation session, with no intermediate confirmation step — the act of dispensing is treated as an authoritative clinical decision that takes immediate effect on stock calculations. This design provides a complete, transaction-level audit trail of all supplement movements, ensuring that the available stock figures presented in the interface always reflect the actual dispensing history of each batch rather than a manually maintained count.

The expiration status of each batch is surfaced visually using a traffic light indicator: batches expiring within one month are flagged in red, those expiring within three months in amber, and those with a longer remaining shelf life in green. This indicator appears both in the Inventory view and within the Supplement Dispensing card during consultation, ensuring that time-sensitive stock information is available at the moment it is needed for clinical decisions.

OCR Pipeline



Step 1 — OCR Extraction

The workflow begins with the user uploading a supplement label image through the frontend interface. The image is submitted to the Python microservice, where PaddleOCR performs text extraction and returns the raw extracted text to the frontend. PaddleOCR was selected for this role on the basis of the benchmark evaluation described in Section 2.3, specifically its 98.31% coverage rate and layout-aware architecture suited to the dense tabular structures of nutrition panels.

The raw OCR output is returned to the frontend before any further processing, allowing the user to review it at Checkpoint 1. This review step is a deliberate design decision: OCR output from real-world supplement labels frequently contains minor character-level errors — misread digits, merged words, or garbled units — that are easier for a human reviewer to identify and correct than for a language model to infer from context alone. Allowing the user to correct the raw text before it is passed to the structuring stage reduces the risk of hallucinated or misassigned nutritional values in the downstream output.

Step 2 — Structure Text

Once the user confirms or corrects the raw OCR text at Checkpoint 1, the text is submitted to GPT-4o-mini, which structures the content into the standardised nutritional schema used throughout NutriFusion — mapping ingredient names, nutrient values, and units to the appropriate fields for `nutritional_info_per_100g` and `nutritional_info_per_serving`. GPT-4o-mini was selected for this structuring task over larger models on the basis of its substantially lower inference cost and latency, with sufficient capability for the structured field extraction task given the relatively constrained and domain-specific nature of nutritional label text.

The structured output is returned to the frontend at Checkpoint 2, where the user can review the parsed fields and correct any values where the language model has hallucinated, misassigned, or omitted data before the search is executed. This second human checkpoint preserves the accuracy of the similarity search by ensuring that the vector embedding is generated from verified data rather than uncorrected model output.

Step 3 — Vectorise and Similarity Search

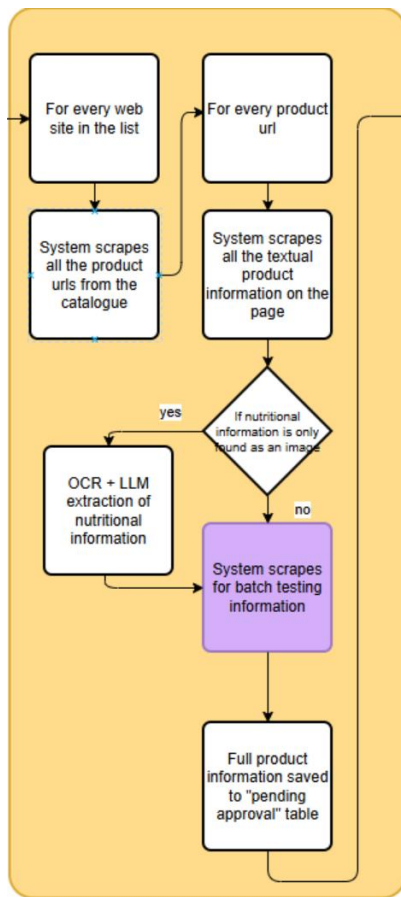
The confirmed structured nutritional data is submitted to the Python microservice, where the BAAI/bge-small-en-v1.5 model generates an embedding vector from the ingredient list and nutritional profile. This vector is then used to query `sss.supplement` via `pgvector` cosine similarity, applying the same `GREATEST` score logic described in Section 4.2.3 — computing similarity against both `vector_100g_ingredient` and `vector_perserving_ingredient` for each supplement in the database and selecting the higher of the two scores as the representative similarity. Results above the 60% threshold are returned as a ranked, paginated list sorted by maximum similarity score, presenting the nutritionist with the closest matching supplements currently in the NutriFusion database.

Design Rationale: Two Human Checkpoints

The inclusion of two human review checkpoints — one after OCR extraction and one after language model structuring — reflects the clinical context in which this feature operates. A similarity search used to inform a supplement substitution decision for an athlete carries a higher cost of error than a general-purpose search. The two-checkpoint design ensures that both the text extraction and the structured interpretation of that text are validated by the user before they influence the search result, balancing automation efficiency with clinical accuracy. This is consistent with the human-in-the-loop principles established in the staging workflow (Section 4.2.2) and with the governance requirements identified in Section 2.6.

Webscraper Pipeline

The Web Scraper and Staging Workflow automates the collection, enrichment, and verification of supplement data from vendor websites, addressing Capability Area 2 of Section 1.5. The pipeline comprises five sequential stages — catalogue scraping, product scraping, OCR enrichment, batch testing verification, and staging insertion — followed by a human review and approval process before any extracted supplement enters the production library, illustrated in Figure below.



Stage 1: Catalogue Scraping

The pipeline begins with catalogue scraping, in which the system navigates to each vendor website registered in `sss.webscraper_catalog_url` and extracts the URLs of all individual product listings from the catalogue page. This stage is handled by a Selenium-based browser automation layer that renders the full page, handles infinite scroll, and dismisses cookie consent popups before passing the rendered HTML to ScrapeGraphAI. ScrapeGraphAI uses GPT-4o as its underlying language model, prompting it to identify and return all absolute product URLs from the catalogue HTML while filtering out non-nutritional items such as accessories or apparel. Pagination is handled automatically, with the scraper continuing to advance through catalogue pages until all product URLs have been collected.

Stage 2: Product Detail Scraping

For each product URL collected in Stage 1, the system navigates to the product page and extracts structured supplement data using ScrapeGraphAI. The extraction prompt instructs the model to map all relevant fields — supplement name, brand, ingredient list, nutritional information per 100g, nutritional information per serving, certifications, and warnings — to a standardised JSON schema. A single product page may yield multiple `ScrapedProduct` objects where a product is available in multiple variants (for example, different flavours or sizes), each of which is treated as a distinct staging entry. The language model approach is adopted here specifically because vendor websites do not follow a consistent HTML structure, and a rule-based scraper would require individual maintenance for each target site.

Stage 3: OCR Enrichment

A significant proportion of vendor product pages present nutritional information as an embedded label image rather than structured HTML text. Where the product scraper detects an image URL in place of parseable nutritional data, the image is passed to the OCR enrichment stage. PaddleOCR extracts the raw text from the label image, and GPT-4o-mini then structures the extracted text into the standardised nutritional schema, filling in missing `nutritional_info_per_100g` or `nutritional_info_per_serving` fields that could not be obtained through text scraping alone. Each enriched product record is tagged with a `Nutrition_Source` field indicating whether its nutritional data was obtained via text scraping ("Scraped") or OCR ("OCR"), providing administrators with provenance information during the staging review.

Stage 4: Batch Testing Verification

Following product and OCR extraction, the pipeline automatically attempts to verify each scraped supplement against all six supported certification databases — Informed Sport, Informed Choice, HASTA, NSF Sport, the Cologne List, and BSCG — using the brand and product name extracted in Stage 2. This stage reuses the same certification search infrastructure described in Section 4.2.5, querying all six databases concurrently via Selenium. The verification result, including the testing organisation and source URL where found, is written directly onto the scraped product record before it is pushed to staging, so that administrators reviewing staged entries can see certification status without performing a separate manual lookup. Where no certification match is found, the batch testing fields are left null rather than asserting a negative result, reflecting that an absence of evidence in the queried databases does not confirm that a product is uncertified.

Stage 5: Staging Insertion

All extracted and enriched product records are inserted into `sss.supplement_staging`, where they are held with `is_reviewed = false` pending administrator review. The staging table mirrors the structure of `sss.supplement` but includes additional fields tracking the source catalog URL (`webscraper_catalog_url_id`) and review status. No vectorisation is performed at this stage — embedding generation is deferred to the approval step to avoid generating vectors for records that may be edited or rejected during review.

Scheduler

The scraping pipeline is configured and managed through `sss.scrapers_schedule_config`, which records whether the scheduler is enabled, the interval in days between runs (defaulting to 14), the running status, and timestamps for the last and next scheduled execution. Administrators can configure the interval, enable or disable the scheduler, and trigger manual runs through the web scraper management interface. The scheduler is implemented using APScheduler within the Python FastAPI service, which executes the full five-stage pipeline as a background job on the configured interval without blocking the main API.

Staging Review and Approval

Once staging entries are populated, administrators review each entry through the staging interface, where they can inspect the extracted data, correct any fields where the scraper or language model produced inaccurate or incomplete output, and either approve or reject the entry. The ability to edit extracted data before approval is a deliberate design decision: it ensures that the human-in-the-loop step is substantive rather than binary, allowing administrators to salvage partially correct extractions rather than discarding them entirely.

When an administrator submits a staging entry for approval, a three-step approval workflow is executed, illustrated in Figure below:



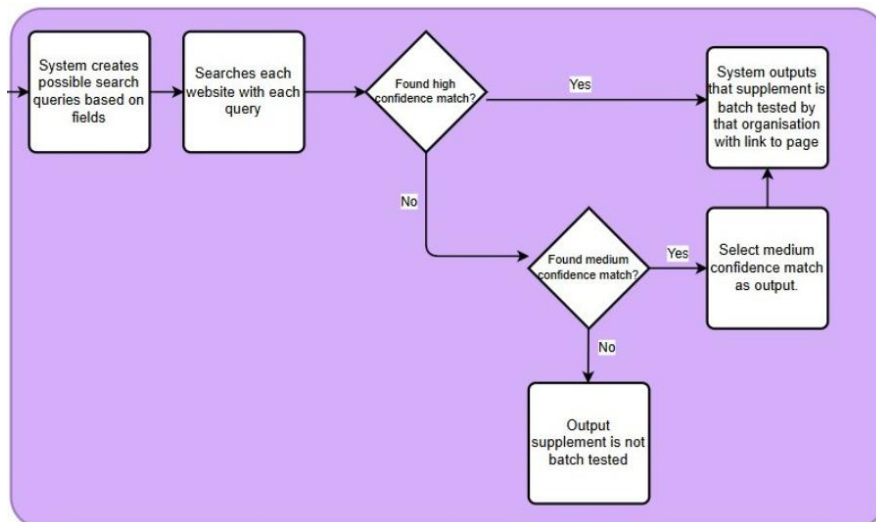
Step 1 — Vectorise: The Python microservice generates 384-dimensional embeddings for the staging entry using the BAAI/bge-small-en-v1.5 model, producing `vector_100g_ingredient` and `vector_perserving_ingredient` vectors from the supplement's ingredient list and nutritional profiles. If vectorisation fails — for example due to missing nutritional data — the approval process is halted and the administrator is notified.

Step 2 — Duplicate Detection: The newly generated vectors are used to query `sss.supplement` via pgvector cosine similarity, returning the top 10 existing supplements with a similarity score of 95% or above. If any candidates are found, a normalised string comparison is performed on supplement name and brand (lowercased, trimmed) against each candidate. If both name and brand match any candidate, the staging entry is flagged as a probable duplicate and the administrator is presented with the option to delete the entry or continue editing it. If no name-and-brand match is found among the high-similarity candidates, the entry is not considered a duplicate and the approval proceeds.

Step 3 — Promotion: If no duplicate is detected, the staging entry is promoted to `sss.supplement` with its generated vectors, `is_reviewed` is set to true on the staging record, and `promoted_to_supplement_id` is populated with the UUID of the newly created supplement record, maintaining a traceable link between the staged source and the production entry.

This three-step workflow ensures that the production supplement library remains free of duplicates introduced by repeated scraping of the same products across different scheduling runs, while preserving the flexibility for administrators to resolve edge cases where the duplicate detection logic produces uncertain results.

Batch Testing Pipeline



Supported Databases and Concurrent Search

Six certification databases are supported: Informed Sport, Informed Choice, HASTA, NSF Sport, the Cologne List, and BSCG. Of these, five support batch identifier lookup (all except the Cologne List, which only supports product name search). All six are queried in parallel using `asyncio.gather()` within the Python FastAPI service, which dispatches concurrent Selenium browser sessions — one per database — rather than querying them sequentially. A pool of six Selenium Chrome driver instances is maintained alive across requests and reused, avoiding the startup latency of initialising a new browser session for each verification request. This concurrency design is motivated by the latency characteristics of the task: each certification database search may take tens of seconds due to external site response times, and sequential querying across six databases would produce unacceptably long response times in practice.

Each database has a custom interaction function that handles its specific page structure — navigating to the search interface, dismissing cookie consent dialogs, accepting terms and conditions where required (as in the case of the Cologne List), submitting the search query, and waiting for results to render before extraction. This site-specific handling is necessary because the certification databases do not provide public APIs, and each presents a distinct search interface that cannot be navigated through a generic browser automation approach.

Search Term Strategy

For each database, the system generates and attempts two to three search term variations in sequence: the brand name alone, the brand name combined with the product name, and where applicable the brand name combined with the product name and any variant descriptor. This progressive narrowing strategy ensures that a match is found at the broadest level if a more specific query returns no results, and reduces the risk of missing a certification record due to minor discrepancies between the product name as entered and the name as recorded in the certification database.

Confidence Scoring

Each database search returns a result tagged with a confidence level — high, medium, or low — based on the nature of the match found. A high confidence result is assigned when a direct link to a specific certified product page is returned, indicating a precise match between the search query and a certified record. A medium confidence result is assigned when the search returns a listing that plausibly matches the query but without a direct product-level link. A low confidence

result is assigned when the search returns no usable match. As illustrated in Figure X, the system outputs a high confidence match where one is found; if no high confidence match is available, a medium confidence match is selected as the output; if neither is found, the supplement is reported as not verified in that database. This tiered approach allows the system to surface the best available evidence rather than returning a binary verified or not-verified result that would discard potentially useful partial matches.

The `build_verification_summary()` function collates the results across all six databases and returns a structured response indicating overall verification status (`is_verified`), the list of databases in which a match was found (`found_websites`), and direct links to the relevant certification records (`quick_links`) where available.

Batch Identifier Extraction

For input modes that accept a batch or lot number — whether entered as text or extracted from an uploaded image — the system applies a two-stage extraction strategy. In the first stage, a set of regular expressions matches common batch and lot number formats, including patterns such as "Batch: XXXXX", "Lot #: XXXXX", and alphanumeric identifiers like "BN108446". Date strings and telephone numbers are explicitly filtered out at this stage to avoid false positives. If the regex stage returns no result or produces a low-confidence match, a language model fallback is invoked to interpret the text and identify the batch identifier from context. The extraction result includes the identified batch identifier, its type classification (batch, lot, certification number, or unknown), a confidence level, and any alternative candidate identifiers found during extraction.

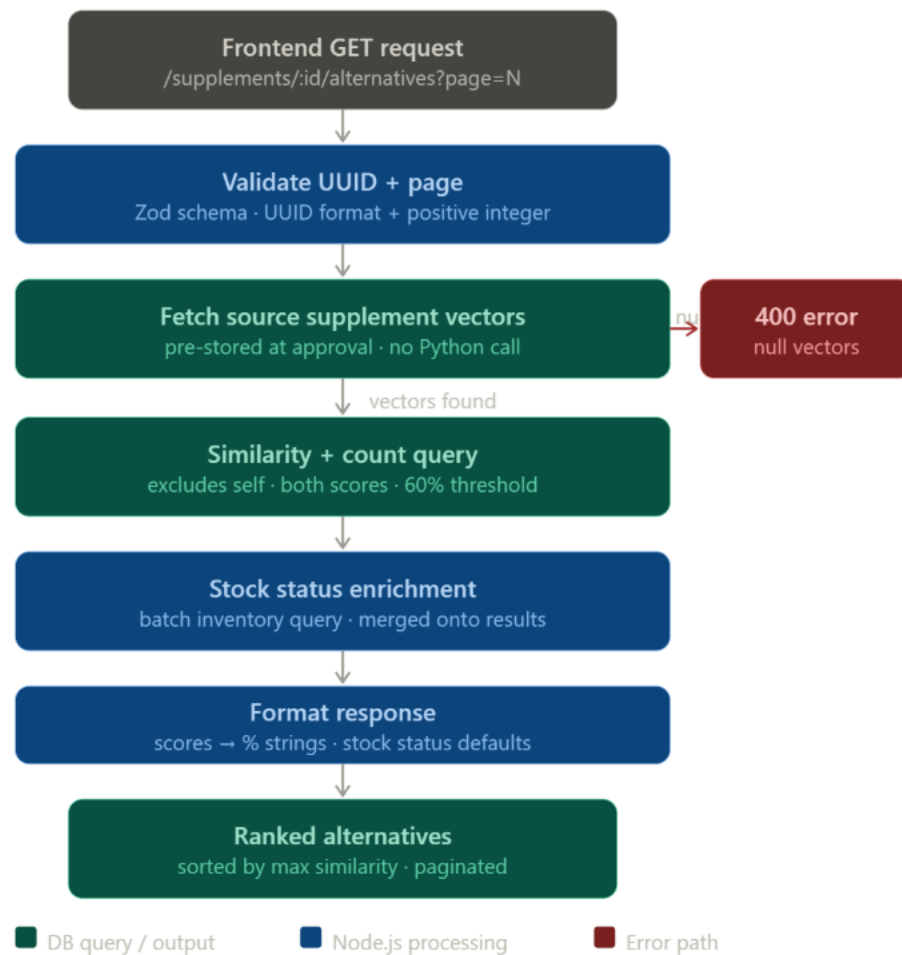
For image-based inputs, PaddleOCR first extracts the raw text from the uploaded image before the batch identifier extraction pipeline is applied to the resulting text, following the same OCR process described in Section 4.2.4.

Latency Management

As described in Section 3.1.2, certification database searches can take up to six minutes due to the response times of external sites. To accommodate this latency within the platform's request architecture, batch verification requests from the frontend are routed through a Next.js API proxy that forwards them to the Node.js backend, where the socket timeout is extended to 360 seconds before the request is forwarded to the Python microservice. This three-hop routing architecture was introduced specifically to work around the non-configurable timeout enforced by Next.js on its own outbound fetch calls, and represents a deliberate infrastructure decision to support the latency profile of this feature without compromising the responsiveness of other platform operations.

Supplement Similarity Search

The Supplement Similarity Search identifies nutritionally and compositionally similar supplements within the NutriFusion database, given a known supplement as a reference. It addresses Capability Area 3 of Section 1.5 and is accessible from the detail view of any supplement in the library. The full query workflow is illustrated in Figure below.



Embedding Model and Dual Vector Strategy

Each supplement in the database is represented by two 384-dimensional embedding vectors, generated by the BAAI/bge-small-en-v1.5 sentence transformer model hosted in the Python microservice (Xiao et al., 2023). This model was selected for its compact size relative to its semantic performance, its suitability for CPU inference without GPU dependency, and its production-stable availability via the HuggingFace sentence-transformers library.

The dual vector strategy generates one embedding per nutritional basis: `vector_100g_ingredient` encodes the supplement's ingredient list combined with its nutritional information per 100g, and `vector_perserving_ingredient` encodes the ingredient list combined with its nutritional information per serving. The rationale for maintaining two separate vectors rather than a single combined embedding is that supplement products vary significantly in how their nutritional data is declared — some products provide only per-100g data, some only per-serving data, and some provide both. A single vector that concatenated both bases would be sparse for products with only one basis, introducing noise into the similarity calculation. By maintaining two independent vectors, the similarity search can compare products on whichever nutritional basis is most meaningfully populated, and take the better of the two scores as the overall similarity.

Both vectors are generated at the point of supplement creation or approval and stored directly on `sss.supplement`. At query time, no call to the Python microservice is required — the pre-stored vectors are retrieved from the database and used directly in the similarity query, keeping query latency within the database layer.

Query Workflow

The similarity search is triggered by a GET request to `/api/SSS/supplements/:id/alternatives?page=N`. The request is validated by a Zod schema that enforces a valid UUID format for the supplement identifier and a positive integer for the page number. The backend then fetches the two pre-stored vectors for the reference supplement from `sss.supplement`. If both vectors are null — indicating that the supplement was created without sufficient nutritional data to generate embeddings — the request returns a 400 error, as a meaningful similarity search cannot be performed without a valid embedding.

Where vectors are available, a combined similarity and count query is executed against `sss.supplement` using `pgvector`'s cosine distance operator (`<=>`). The query computes cosine similarity scores for both `vector_100g_ingredient` and `vector_perserving_ingredient` for every other supplement in the database (excluding the reference supplement itself), and selects the `GREATEST` of the two scores as the representative similarity score for each candidate. Only candidates with a `GREATEST` score of 60% or above are returned, reflecting the threshold calibrated against HPSI's supplement catalogue during development to define operationally meaningful similarity. Results are sorted in descending order of this score and paginated to support efficient retrieval across large catalogues.

Following the similarity query, each result is enriched with current stock status information by performing a batch inventory query — summing ticket quantities against initial quantities for all batches of each candidate supplement — and merging the computed stock status onto the result set. Similarity scores are formatted as percentage strings for display. The final response presents a ranked, paginated list of alternative supplements with their similarity scores and stock availability, giving nutritionists both the nutritional basis for each recommendation and an immediate indication of whether a recommended alternative is currently in stock.

Threshold Rationale

The 60% similarity threshold for alternative recommendations and the 95% threshold used for duplicate detection during staging approval were established empirically by testing the similarity distribution across NutriFusion's supplement catalogue. At 60%, the results include supplements that share a meaningful proportion of their ingredient and nutritional profile with the reference, while excluding products that are too nutritionally dissimilar to serve as practical substitutes. At 95%, the similarity is high enough that two products are very likely to be the same product entered twice — capturing genuine duplicates while avoiding false positives that would suppress legitimate near-identical alternatives.

5.2 AMS — Athlete Management System

The AMS manages athlete profiles and links them to their consultation history.

Dashboard

The Dashboard functions as the operational hub and primary entry point for NutriFusion, consolidating team-wide clinical data and personal scheduling into a single view. It provides a platform-wide summary of active athlete counts alongside the logged-in user's daily totals to

balance broader team context with immediate individual workloads. A shared session list offers a unified view of all team consultations—including athlete names and venues—utilising color-coded statuses like Yellow (Scheduled) and Red (Cancelled) to facilitate rapid identification. Furthermore, the day schedule view utilises a timeline visualisation with a real-time red line indicator for enhanced time awareness, while integrated quick actions allow for direct session booking to minimise navigation overhead.

Consultation sessions progress through a deterministic lifecycle across four states to ensure an unambiguous representation of the system's operational state. Each record is initialised as Scheduled upon creation, indicating the session date has not yet passed. A session transitions to Completed once the clinical cards are successfully saved, while a Cancelled status results from explicit user action via the dashboard or consultation interface. If a scheduled timeslot passes without the session being completed or cancelled, the system automatically transitions the status to Expired. This structured framework ensures that both live dashboard views and historical records remain accurate and actionable for the nutrition team.

Athlete Profile Management

Athlete profiles in NutriFusion are structured across three specialised tables—ams.athlete (base), ams.athlete_registry, and ams.athlete_medical—adhering to 5NF normalisation principles. This architectural separation allows the base, carding, and clinical domains to be updated independently, reducing the risk of partial overwrites to unrelated fields. To ensure total data integrity, profile creation is executed as an atomic transaction using explicit BEGIN, COMMIT, and ROLLBACK statements. If any insertion fails, the entire transaction is reversed, preventing the existence of incomplete records that could disrupt downstream operations like supplement dispensing or session creation.

The data model captures a comprehensive range of personal, sporting, and clinical details, including carding status, medical conditions, and dietary restrictions. Data integrity is enforced at the database level through CHECK constraints on fields such as gender and sport duration. Within the Athlete Detail View, nutritionists can access these records alongside dedicated tabs for managing active consultations and reviewing historical session data. Relationship management is handled via ams.nutritionist_athlete_mapping, which supports many-to-many associations between athletes and staff. While standard users can only assign athletes to themselves at creation, administrators maintain the authority to manage assignments across the entire platform.

5.3 Consultation Module

The Athlete Management System (AMS) is the primary user-facing layer of NutriFusion, addressing Capability Area 5 of Section 1.5. It provides nutritionists with a unified environment for managing athlete profiles, conducting structured consultation sessions, tracking longitudinal consultation history, and coordinating daily workload through a shared dashboard.

Consultation Session Management

Consultation sessions are the primary clinical record unit in NutriFusion, linking nutritionists to athletes through the consultation.sessions table. To prevent duplicate entries, the system checks the is_scheduled_booking flag; if a scheduled session already exists for an athlete, the user is prompted to continue that record rather than creating a new one.

The Nine-Card Framework

The interface utilises a fixed nine-card sequence that mirrors a natural clinical workflow. Each card includes a Previous Session Comparison tab, allowing nutritionists to track progress against the most recent data without leaving the active session.

Section 1: Context & Physical Assessment

1. Consultation Details: Captures administrative metadata, clinical objectives, and follow-up dates.
2. Anthropometry: Records physical measurements (height, weight, skinfolds). Stored generated columns in the database automatically compute BMI and skeletal muscle mass percentages upon data entry.
3. Medical History: Centralises bowel regularity, hydration (with reference urine charts), puberty, and menstrual cycle data across four dedicated tables.

Section 2: Lifestyle & Requirements

4. Training Schedule: Tracks exercise programs, competition dates, and Rating of Perceived Exertion (RPE). Training duration is auto-computed based on start/end times.
5. Meal Log: Logs typical dietary intake (macronutrient ranges) and sleep metrics, reflecting the clinical link between nutrition and recovery.
6. Nutrition Requirements: Provides the analytical basis for diagnosis by comparing estimated intake against required grams per kilogram of body weight.

Section 3: Diagnosis & Intervention

7. Nutrition Diagnosis & Summary: Records the primary diagnosis, intervention plans, and specific macronutrient review notes.
8. Actionables: Manages follow-up tasks by recording item descriptions, owners, and due dates.
9. Supplement Dispensing: Directly integrates with the SSS inventory. Dispensing a supplement triggers an inventory_ticket, recalculates batch stock, and utilises the traffic-light system to alert nutritionists to expiring stock at the point of prescription.

5.4 Admin Module

Sub-module	Route Prefix	Key Functionality
Users	/api/admin/users	Create, view, edit, deactivate user accounts; role assignment
Sports & Coaches	/api/admin/sports-coaches	Manage sport categories and coach records
Audit Log	/api/admin/audit	Append-only log of user actions with timestamps

The Administration module exposes four management capabilities, all gated behind role-based access control so that only users with administrator privileges can reach them.

User Management

User accounts in NutriFusion are stored in auth.users, which carries the user's email address, bcrypt-hashed password, assigned role, and active status. Administrators can create new user

accounts, update existing credentials and role assignments, and deactivate accounts without deleting them — preserving the referential integrity of audit records and consultation histories that link to a user's identifier. Every write operation executed through the user management interface is captured by the audit trigger described in Section 4.1.9, recording the responsible administrator, the affected record, and JSONB representations of the old and new field values in `audit.audit_log`. This log is surfaced within the administration interface itself, allowing administrators to review the full change history of any user account without leaving the platform.

The role values supported by the system are: `IT_ADMIN`, `ADMIN`, `NUTRITIONIST`, `COACH`, `ATHLETE`, and `DASHBOARD`. Role assignment determines which routes and data a user can access across the platform, though full route-level enforcement of role distinctions beyond the administrator boundary is scoped for a future development phase, as noted in Section 8.3.

Sports and Coach Management

Sports and coach records serve as the reference data that populates dropdown fields across the athlete management workflow. The sports catalogue is maintained in `ams.sport_lookup`, and the coach catalogue in `ams.coach`, both following the dynamic lookup table pattern described in Section 4.1.4 — new entries can be added and existing entries retired via the `is_active` flag without disrupting records that already reference them. Administrators manage both catalogues through the administration interface, ensuring that the options available to nutritionists when creating and updating athlete profiles remain accurate and up to date.

Web Scraper Configuration

Access to the web scraper management interface is restricted exclusively to administrators. Through this interface, administrators register target vendor catalogue URLs in `sss.webscraper_catalog_url`, configure the automated scraping schedule in `sss.scrapper_schedule_config`, and initiate manual scraping runs. The schedule configuration records whether the scheduler is enabled, the interval in days between runs, and the timestamps for the last and next scheduled execution. Restricting scraper configuration to administrators ensures that the automated supplement data collection pipeline — and by extension, the staging area it populates — is operated under accountable oversight, consistent with the governed ingestion requirements identified in Section 2.6.

Athlete Assignment

When creating a new athlete profile, administrators can assign the athlete to any nutritionist on the platform. Standard nutritionist users may only assign athletes to themselves. This distinction is implemented at the controller level: the assignment field in the athlete creation request is validated against the requesting user's role, and any attempt by a non-administrator user to specify a nutritionist other than themselves is rejected with a 403 response. This design supports organisational management of athlete caseloads — for example, when a new cohort of athletes must be distributed across the nutrition team — while preserving the self-service capability for nutritionists managing their own athletes.

5.5 Auth Module

Endpoint	Method	Description
<code>/api/auth/login</code>	POST	Validate credentials, generate OTP, send via email
<code>/api/auth/verify-code</code>	POST	Validate OTP, issue JWT (24h expiry)

Endpoint	Method	Description
/api/auth/logout	POST	Client-side token discard (server stateless)
/api/auth/me	GET	Return authenticated user profile from JWT

The Authentication module implements the two-factor authentication flow and session management infrastructure required to satisfy the access control requirements of Section 2.6. It is built on three components: password hashing, JWT-based session tokens, and a six-digit email verification code system.

Password Hashing

User passwords are hashed using bcrypt with a salt round value of 10, corresponding to a cost factor of $2^{10} = 1024$ iterations. The resulting hash is stored in `auth.users.password_hash`; the plaintext password is never persisted. The bcrypt cost factor was chosen to balance verification latency against brute force resistance at the current scale of the platform, and can be increased in future iterations without requiring existing password hashes to be migrated — bcrypt encodes the cost factor within the hash string itself, allowing the verification function to apply the correct number of iterations regardless of when the hash was generated.

Two-Factor Authentication Flow

Authentication proceeds in two sequential steps. In the first step, the user submits their email address and password to `POST /api/auth/login`. If the credentials are valid, a cryptographically random six-digit code is generated and stored in `auth.verification_codes` alongside the associated user identifier, a purpose field, an attempt counter initialised to zero, and an expiry timestamp set to ten minutes from the time of generation. The code is then dispatched to the user's registered email address via Nodemailer. The response returned to the client contains only a masked representation of the email address, confirming that the code has been sent without exposing the full address.

In the second step, the user submits the six-digit code to `POST /api/auth/verify-code`. On each submission, the attempt counter on the `verification_codes` row is incremented. If the submitted code matches the stored value and has not expired, the verification record is deleted and a JWT is issued. If the number of incorrect attempts reaches the threshold defined by the `MAX_VERIFICATION_ATTEMPTS` environment variable, the verification record is deleted entirely, invalidating the code and requiring the user to re-initiate the login flow. This application-level attempt limit operates independently of the IP-based rate limiting described below — the attempt counter tracks failures per code instance, not per IP address, ensuring that the lockout applies even when requests originate from different network locations.

JWT Session Tokens

Upon successful verification, a JSON Web Token is issued containing the payload fields `userId`, `email`, and `role`. The token expiry is read from the `JWT_EXPIRY` environment variable rather than hardcoded, allowing the expiry window to be adjusted without a code change. Standard `iat` (issued at) and `exp` (expires at) claims are added automatically by the `jsonwebtoken` library. All subsequent requests to protected endpoints must include the token in the Authorisation header as a Bearer credential; the authentication middleware validates the token signature and expiry before attaching the decoded payload to the request object for downstream use.

Logout is implemented as a stateless operation: the client discards the token locally, and no server-side invalidation is performed. The `auth.user_sessions` table, which records token hashes, IP addresses, user agents, and expiry timestamps, exists as infrastructure for a future token

blacklisting implementation but is not actively used for invalidation in the current release. The adoption of Redis-based token blacklisting to support active session revocation is identified as a planned future enhancement in Section 8.3.

Rate Limiting

To protect the authentication layer against brute force and denial-of-service attempts, rate limiting is applied independently to each authentication endpoint using express-rate-limit, keyed by IP address. The limits are as follows: POST /api/auth/register accepts three requests per hour; POST /api/auth/login accepts five requests per fifteen-minute window; POST /api/auth/verify-code accepts three requests per minute; and POST /api/auth/resend-code accepts two requests per five minutes. Each limiter is configured with standardHeaders: true and legacyHeaders: false, making the rate limit state visible to clients via RFC 6585-compliant RateLimit-* response headers rather than the deprecated X-RateLimit-* headers. The independent limiter configuration ensures that a burst of activity on one endpoint does not affect the limits applied to another, preserving the usability of the resend and verification flows even under high login load.

5.6 OCR Services

Endpoint	Method	Description
/ocr/scan-label	POST	Accept image, run PaddleOCR, parse with GPT-4o-mini
/certifications/verify	POST	Use Playwright to check batch against 6 certification databases
/embeddings/generate	POST	Generate 384-dim vector for a supplement record
/scraper/run	POST	Execute a web scrape job and return staged results

6. Database Design

All the sections mentioned are referenced from our final report.

6.1 Schema Overview

The database is organised into five PostgreSQL schemas — sss, ams, consultation, auth, and audit — each corresponding to a distinct operational domain of the platform. This schema-level separation serves both governance and maintainability purposes: access permissions can be granted at the schema level to restrict which application roles can read or write to which domain, and schema boundaries provide a clear organisational boundary that reflects the modular architecture of the application layer above it. Two PostgreSQL extensions are enabled at the database level: pgcrypto, which provides cryptographic functions used in authentication, and vector (pgvector), which enables the storage and cosine similarity querying of 384-dimensional embedding vectors used for supplement similarity search.

Table	Module	Description
audit_log	Audit	Stores audit trail of data changes (insert, update, delete) with old/new values and user tracking
scraper_schedule_config	SSS	Configuration for automated web scraper scheduling and execution tracking
webscraper_catalog_url	SSS	Stores product catalog URLs used by the web scraper
supplement_staging	SSS	Temporary staging table for scraped/manual supplements pending review and approval
supplement	SSS	Master table for approved supplements with nutritional info and metadata
batch_testing_org_lookup	SSS	Lookup table for supplement batch testing organisations
supplement_status_lookup	SSS	Lookup table for supplement statuses (e.g. batch tested, discontinued)
supplement_packaging_form_lookup	SSS	Lookup table for supplement packaging forms (e.g. tub, tablet)
inventory_batch	SSS	Stores supplement batch inventory details including stock, price, and expiry
batch_stock_status_lookup	SSS	Lookup for batch stock status (available, low, out of stock)
inventory_ticket	SSS	Tracks inventory movements linked to prescriptions and batch usage
ticket_status_lookup	SSS	Lookup for inventory ticket statuses (booked, completed, returned)
Athlete	AMS	Core athlete profile including demographics and sport

Table	Module	Description
audit_log	Audit	Stores audit trail of data changes (insert, update, delete) with old/new values and user tracking
scraper_schedule_config	SSS	Configuration for automated web scraper scheduling and execution tracking
Coach	AMS	Stores coach details and associated sport
coach_athlete_mapping	AMS	Maps coaches to athletes (many-to-many relationship)
sport_lookup	AMS	Lookup table for sports
nutritionist_athlete_mapping	AMS	Maps nutritionists to athletes
athlete_registry	AMS	Tracks athlete carding status, approval periods, and registry data
nutritionist	AMS	Stores nutritionist profiles
athlete_medical	AMS	Stores athlete medical history and conditions
user_athlete_pins	AMS	Stores user-pinned athletes for quick access
sessions	Consultation	Core consultation session linking athlete and nutritionist
type_of_consult_lookup	Consultation	Lookup for consultation types
session_open_item	Consultation	Tracks open tasks or follow-ups within a session
open_item_status_lookup	Consultation	Lookup for open item statuses
session_prescription	Consultation	Stores prescribed supplements and dosage details
session_prescription_note	Consultation	Stores additional notes related to prescriptions
session_anthropometry	Consultation	Stores body measurements and calculated metrics (BMI, etc.)
session_training	Consultation	Stores training-related information and performance context
session_training_schedule	Consultation	Detailed day-by-day training schedule for a session
session_meal	Consultation	Stores overall meal-related notes for a session
session_meal_log	Consultation	Detailed meal logs including macronutrient ranges

Table	Module	Description
audit_log	Audit	Stores audit trail of data changes (insert, update, delete) with old/new values and user tracking
scraper_schedule_config	SSS	Configuration for automated web scraper scheduling and execution tracking
session_sleep	Consultation	Tracks sleep duration, quality, and notes
session_note	Consultation	Stores nutrition diagnosis, reviews, and intervention notes
consultation_objective_lookup	Consultation	Lookup for consultation objectives
session_nutrition_review	Consultation	Stores dietary intake estimates and nutritional requirements
session_period	Consultation	Tracks menstrual cycle and related health data
session_puberty	Consultation	Tracks puberty and growth-related information
session_bowel_movement	Consultation	Tracks bowel movement patterns and digestive health
session_hydration	Consultation	Tracks hydration status and water intake
users	Auth	Stores system users, roles, and authentication data
verification_codes	Auth	Stores temporary verification codes for login, reset, or email verification
user_sessions	Auth	Tracks active user sessions and authentication tokens

6.2 Database Functions

Function Name	Schema	Primary Purpose	Deployment / Use Case
uuid_generate_v7()	public	Generates a time-sortable Version 7 UUID. This ensures global uniqueness while embedding a millisecond-precision timestamp to maintain efficient, sequential B-Tree indexing.	Set as the DEFAULT value for the id column across all tables in the database.
audit_trigger_func()	audit	Acts as the system's immutable ledger. It captures INSERT, UPDATE, and DELETE events,	Attached as an AFTER trigger to all core operational tables (e.g.,

		explicitly checks if data has actually changed, and logs the user ID along with the JSONB snapshots of the old and new values.	<i>ams.athlete, sss.supplement, consultation.sessions).</i>
update_modified_column()	public	Automatically refreshes the timestamp of a modified record. It compares the old and new row data and only updates the <i>updated_at</i> column to <i>NOW()</i> if genuine modifications are detected.	Attached as a BEFORE UPDATE trigger on almost every table containing an <i>updated_at</i> column.
touch_parent_timestamp()	public	Executes the cascading state synchronisation protocol. It accepts dynamic arguments to locate a parent record via a foreign key and updates that parent's <i>updated_at</i> timestamp when a child record is modified.	Attached as an AFTER trigger on child tables (e.g., modifying <i>session_meal_log</i> automatically touches <i>session_meal</i>)

6.3 Key Design Decisions

Normalisation to Third Normal Form

The database schema is designed to conform to Third Normal Form (3NF), eliminating all redundancy by ensuring that every fact is recorded in exactly one place and that all join dependencies are explicitly represented through foreign key relationships. In practice, this means that supplement nutritional information is stored once in *sss.supplement* and referenced across the platform, rather than being duplicated into *athlete* or *consultation* records. Similarly, *athlete* medical and registry data are separated from the base *athlete* record into *ams.athlete_medical* and *ams.athlete_registry* respectively, each linked via foreign key to *ams.athlete*. This decomposition ensures that updates to shared data propagate consistently without the risk of partial or conflicting records.

UUIDv7 Primary Keys

All tables in the database use UUIDs as primary keys, generated via the *uuid_generate_v7()* function. UUIDv7 was selected over both auto-incrementing integers and UUIDv4 for two reasons. First, UUIDs eliminate the collision risk that arises when integrating with external systems or merging data from multiple sources, a critical consideration given HPSI's existing data infrastructure. Second, UUIDv7's time-sortable structure, in which the most significant bits encode a millisecond-precision timestamp, preserves B-tree index locality in PostgreSQL. Unlike UUIDv4, which is randomly structured and therefore causes random index page splits on insertion, UUIDv7 inserts are approximately sequential, maintaining index compactness and improving query performance as the database grows.

Dynamic Lookup Tables

Reference data across the platform, including supplement packaging forms, supplement statuses, batch stock statuses, batch testing organisations, ticket statuses, sports, and consultation types, is managed through dedicated lookup tables rather than hardcoded PostgreSQL ENUM types. Each lookup table follows a consistent structure: a UUIDv7 primary key, a text value field, and an *is_active* boolean flag that supports soft deletion without removing records that may be referenced by historical data. This approach allows new reference values to be added or retired dynamically

without requiring schema migrations or application downtime, which is particularly important in an operational environment where new supplement categories or consultation types may need to be introduced without disrupting live workflows.

JSONB and Array Column Types

Supplement nutritional data presents a structuring challenge: the set of nutrients recorded for any given product varies significantly across products and vendors, making a fixed relational schema impractical. NutriFusion addresses this through the use of PostgreSQL's JSONB column type for three fields on *sss.supplement*: *supplement_ingredient* (an array of ingredient strings), *nutritional_info_per_100g* (a key-value object mapping nutrient names to declared amounts), and *nutritional_info_per_serving* (the equivalent per-serving data). JSONB stores data in a decomposed binary format that supports efficient indexing via GIN (Generalised Inverted Index), enabling fast searches within the nutritional content of supplements without requiring full-table scans. Product source URLs are stored as a PostgreSQL TEXT[] array in *product_source_url*, reflecting that a supplement may be listed across multiple vendor pages and that this field does not require the queryability that JSONB provides.

Inventory Ticketing System

Stock availability in NutriFusion is not maintained as a static quantity field on the batch record. Instead, every supplement dispensing and reservation event is recorded as a row in *sss.inventory_ticket*, which links an inventory batch (*inventory_batch_id*) to an athlete (*athlete_id*) and a prescription (*prescription_id*), carrying a quantity and a ticket status. Available stock for any batch is therefore calculated dynamically as the difference between *batch_initial_quantity* on *sss.inventory_batch* and the sum of quantities across active tickets for that batch. This approach provides a complete, auditable history of all supplement movements at the batch level, rather than a snapshot of current stock that would conceal the transactional history underlying it. The implementation of inventory management within the SSS is described further in Section 4.2.1.

No ORM (Object-Relational Mapping)

The backend architecture intentionally bypasses traditional ORMs in favour of raw, parameterised SQL throughout the platform. This design choice guarantees query transparency, prevents the generation of unoptimised N+1 queries, and provides explicit control over the complex JOIN operations required by the highly normalised database schema.

6.4 Security, Compliance & Automation

Immutable Audit Trail

All write operations across the platform are recorded in the *audit.audit_log* table, which captures the user responsible, the affected table and record, the nature of the operation, and JSONB representations of the old and new values with a precise timestamp. A robust database-level trigger (*audit.audit_trigger_func()*) guarantees comprehensive clinical compliance by firing automatically on all INSERT, UPDATE, and DELETE operations. The function intelligently compares row states to filter out redundant saves and retrieves the executing application user from the session-level setting *app.current_user_id*. Implementing this at the trigger level ensures that write operations are captured regardless of the application pathway. Furthermore, the audit logs are strictly insert-only; no update or delete privileges are granted on that table, providing a tamper-evident ledger.

Cascading State Synchronisation

To maintain accurate historical records without relying on application-level logic, a custom *touch_parent_timestamp* trigger automates state tracking. Any modification to an underlying child record (such as adding a meal log or updating anthropometry) automatically propagates upward, refreshing the *updated_at* timestamp of the master consultation session.

6.5 Database-Level Computation

To minimise redundant application-level processing and guarantee data consistency, complex clinical metrics are calculated directly within the database layer. This ensures that derived values are always consistent with their source data regardless of which application pathway writes the underlying fields.

- **Stored Generated Columns:** Deterministic metrics that rely on single-row data are computed and stored upon row insertion or modification within the core consultation tables. For example, in *consultation.session_anthropometry*, four metrics are computed as stored columns: BMI, target BMI, fat mass percentage, and skeletal muscle mass percentage. Similarly, in *consultation.session_training_schedule*, training duration in hours is computed dynamically from the difference between *time_start* and *time_end*, including logic to handle sessions that cross midnight.
- **Dynamic Views:** Metrics requiring cross-table aggregation or complex conditional logic are pushed to read-only PostgreSQL Views (e.g., *v_session_nutrition_review*). These views dynamically calculate variable metrics on the fly, including resting metabolic rates and total energy expenditure, adjusting for specific athlete parameters such as gender, height, and physical activity levels.

6.6 Database Handover Files

In the *Nutrifusion_database_handover_v2.zip*, one will find:

File Name	Description
aws_handover_full_final_Test.dump (Not recommended, Testing data)	The team's test database dump encompassing both the schema structure and the entire local dataset (including test athletes, staged supplements, and consultation logs). Unlike the pure SQL files, this .dump file must be executed using the pg_restore utility. It is designed for spinning up exact replica testing environments rather than deploying a clean production instance.
aws_handover_schema_final.sql (Recommended)	The foundational database blueprint. This script contains the pure structural commands required to build the empty database. It creates the schemas, tables, constraints, indexes (including GIN and ivfflat), and database-level trigger functions (such as the audit ledger and cascading timestamps). It also explicitly enables required PostgreSQL extensions like pgvector and pgcrypto. It does not contain any operational data.
Dbdiagram_Nutrifusion_v5 (Recommended)	The complete Entity-Relationship Diagram (ERD) mapping out the final Phase 5 database architecture. This serves as the primary visual reference for the engineering team, illustrating the highly normalised table structures, foreign key relationships, and the logical separation of the five core schemas (AMS, SSS, Consultation, Auth, and Audit). This gives a complete picture of the database relations and provides a platform for design and

	prototyping. The code can be copied over to https://dbdiagram.io/. An example of how the tables are arranged can be found https://dbdiagram.io/d/HPSI-v5-69af8bd1cf54053b6f465f87
insert_reference_lookup_data_final.sql (Reccomended)	The dynamic reference data seeding script. To be executed immediately after the schema is built, this file populates the foundational lookup tables (e.g., sport types, consultation objectives, supplement packaging forms). It relies on the database's native <code>uuid_generate_v7()</code> function to dynamically assign primary keys upon insertion, ensuring the application dropdowns and configurations function on a fresh deployment.
migration_reference_lookup_data_final.sql (Not recommended, Reference file)	This data script contains the hardcoded UUIDs exported directly from the local prototype environment before transitioning to the dynamic insert method. It is preserved for historical reference or strict one-to-one migration testing. <i>(For a fresh deployment, <code>insert_reference_lookup_data_final.sql</code> is the preferred script).</i>

7. API Reference

For more details, you may refer to the swagger.

7.1 SSS Endpoints

Inventory

Method	Endpoint	Auth	Summary	Request Body	Parameters	Responses
GET	/api/SSS/batches	No	List or search inventory batches (paginated)	None	PageParam, SearchParam	200 OK (Paginated list) 400 Bad Request 500 Internal Server Error
POST	/api/SSS/batches	Yes	Create a new inventory batch	supplement_id (UUID) batch_number (string) batch_initial_quantity (int ≥1) batch_price (number ≥0) batch_expiration_date (date) batch_manufacture_date (date)	None	201 Created 400 Bad Request 409 Conflict 500 Internal Server Error
PATCH	/api/SSS/batches/{id}	Yes	Update batch (partial fields allowed)	supplement_id (UUID) batch_number (string) batch_initial_quantity (int) batch_price (number) batch_expiration_date (date) batch_manufacture_date (date)	id (UUID, required)	200 OK 400 Bad Request 404 Not Found 409 Conflict 500 Internal Server Error
DELETE	/api/SSS/batches	Yes	Delete one or more batches (bulk)	ids (array of UUID, min 1)	None	200 OK 400 Bad Request 409 Conflict (if batches have tickets) 500 Internal Server Error

Staging

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/SSS/staging-supplements	✗ No	List staging supplements (unreviewed only, paginated)	None	PageParam	200 OK 400 Bad Request 500 Internal Server Error
GET	/api/SSS/staging-supplements/{id}	✗ No	Get staging supplement details	None	id (UUID)	200 OK 400 Bad Request

						404 Not Found 500 Internal Server Error
PATCH	/api/SSS/staging-supplements/{id}	✔ Yes	Update staging supplement (partial)	supplement_name (string) supplement_description (string) supplement_packaging_form_id (UUID) supplement_status_id (UUID) supplement_ingredient (array) other optional fields	id (UUID)	200 OK 400 Validation Error 404 Not Found 500 Internal Server Error
DELETE	/api/SSS/staging-supplements	✔ Yes	Delete staging supplements (bulk)	ids (array of UUID, min 1)	None	200 OK 400 Bad Request 500 Internal Server Error
POST	/api/SSS/staging-supplements/approve	✔ Yes	Approve staging supplements (bulk)	ids (array of UUID, min 1)	None	200 OK (all success) 207 Multi-Status (partial success) 400 All failed / validation error 500 Internal Server Error
GET	/api/SSS/staging-lookups	✗ No	Get lookup values for staging dropdowns (packaging forms & statuses)	None	None	200 OK 500 Internal Server Error

Catalog

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/SSS/catalog-urls	✗ No	List catalog URLs (paginated)	None	PageParam	200 OK 400 Bad Request 500 Internal Server Error
GET	/api/SSS/catalog-urls/{id}	✗ No	Get catalog URL details	None	id (UUID)	200 OK 404 Not Found 500 Internal Server Error

POST	/api/SSS/catalog-urls	✔ Yes	Create catalog URL	product_catalog_website (string, required)is_active (boolean, optional)	None	201 Created 400 Validation Error 409 Duplicate 500 Internal Server Error
PATCH	/api/SSS/catalog-urls/{id}	✔ Yes	Update catalog URL	product_catalog_website (string)is_active (boolean)	id (UUID)	200 OK 400 Validation Error 404 Not Found 409 Duplicate 500 Internal Server Error
DELETE	/api/SSS/catalog-urls	✔ Yes	Delete catalog URLs (bulk)	ids (array of UUID)	None	200 OK 400 Validation Error 500 Internal Server Error

Scraping

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
POST	/api/SSS/scraping/start	✗ No	Start web scraping job (background)	catalog_url_ids (array of UUID, optional)	None	202 Accepted 400 No valid catalogs 500 Failed to start
GET	/api/SSS/scraping/schedule	✗ No	Get scraping scheduler configuration	None	None	200 OK
PATCH	/api/SSS/scraping/schedule	✔ Yes	Update scraping scheduler configuration	scheduler settings (varies)	None	200 OK 400 Validation Error 500 Internal Server Error

Supplements

Meth od	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/SSS/supplements	✗ No	List/search supplements (paginated, ranked search)	None	PageParam SearchParam	200 OK 400 Bad Request 500 Internal Server Error
GET	/api/SSS/supplements/{id}	✗ No	Get supplement details + stock + batches	None	id (UUID) batchPage (query)	200 OK 400 Invalid ID 404 Not Found 500 Internal Server Error
GET	/api/SSS/supplements/{id}/alternatives	✗ No	Get similar supplements	None	id (UUID) page (query)	200 OK 400 No vectors

			(vector similarity)			404 Not Found 500 Internal Server Error
POST	/api/SSS/supplements	✔ Yes	Create supplement (with vectorisation)	supplement_name (string) supplement_packaging_form_id (UUID) supplement_status_id (UUID) supplement_ingredient (array) nutritional_info_per_100g (object) OR nutritional_info_per_serving (object) batch_testing_org (conditional)	None	201 Created 400 Validation Error 409 Conflict 500 Internal Server Error
PATCH	/api/SSS/supplements/{id}	✔ Yes	Update supplement (partial)	Any editable fields (14 total) e.g. supplement_name (string) supplement_status_id (UUID) product_source_url (string/array)	id (UUID)	200 OK 400 Validation Error 404 Not Found 500 Internal Server Error
DELETE	/api/SSS/supplements	✔ Yes	Delete supplements (bulk)	ids (array of UUID, min 1)	None	200 OK 400 Validation Error 500 Deletion Failed

Lookups

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/SSS/lookups/packaging-forms	✗ No	Get packaging form options	None	includeInactive (boolean, query)	200 OK 500 Internal Server Error
GET	/api/SSS/lookups/supplement-statuses	✗ No	Get supplement status options	None	includeInactive (boolean, query)	200 OK 500 Internal Server Error
GET	/api/SSS/lookups/batch-statuses	✗ No	Get batch stock status options	None	includeInactive (boolean, query)	200 OK 500 Internal Server Error
GET	/api/SSS/lookups/ticket-statuses	✗ No	Get ticket status options	None	includeInactive (boolean, query)	200 OK 500 Internal Server Error

GET	/api/SSS/lookups/batch-testing-orgs	✗ No	Get batch testing organisations	None	None	200 OK 500 Internal Server Error
-----	-------------------------------------	------	---------------------------------	------	------	-------------------------------------

7.2 AMS Endpoints

Athlete

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/AMS/athletes	✓ Yes	List athletes with pagination and search	None	page (number, query), search (string, query)	200 OK, 400 Bad Request, 500 Internal Server Error
GET	/api/AMS/athletes/{id}/profile	✗ No (should likely be protected)	Get athlete profile details	None	id (string, uuid, path)	200 OK, 400 Bad Request, 404 Not Found, 500 Internal Server Error
PATCH	/api/AMS/athletes/{id}/profile	✓ Yes	Update athlete profile (non-admin)	Object (partial): athlete fields (string/date), registry fields (string/date/boolean), medical fields (string), coach_ids (array of UUID strings)	id (string, uuid, path)	200 OK, 400 Bad Request, 404 Not Found, 409 Conflict, 500 Internal Server Error
PATCH	/api/AMS/athletes/{id}/profile/admin	✓ Yes (Admin)	Update athlete profile (admin)	Object (partial): athlete + registry + medical + coach_ids (array of UUIDs) + nutritionist_ids (array of UUIDs)	id (string, uuid, path)	200 OK, 400 Bad Request, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Server Error
POST	/api/AMS/athletes/complete	✓ Yes	Create full athlete (transactional)	Object (required fields): athlete (string/date), registry (string/date/boolean), medical (string, optional), coach_ids (array of UUIDs, optional)	None	201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error
POST	/api/AMS/athletes/complete/admin	✓ Yes (Admin)	Create full athlete (admin assigns nutritionist)	Object (required fields): athlete + registry + medical + coach_ids (array	None	201 Created, 400 Bad Request, 403 Forbidden, 409 Conflict, 500

				of UUIDs, optional) + nutritionist_id (UUID string, required)		Internal Server Error
POST	/api/AMS/athletes	✔ Yes	Create basic athlete	Object (required): sport_id (UUID), sportsync_id (string), athlete_name_a_bbr (string), gender (enum string), date_of_birth (date string); optional fields (string/date)	None	201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error
DELETE	/api/AMS/athletes	✔ Yes (Admin)	Bulk delete athletes	Object: ids (array of UUID strings, min 1)	None	200 OK, 400 Bad Request, 500 Internal Server Error

Coaches

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/AMS/coaches	✗ No	List all coaches with sport names	None	None	200 OK, 500 Internal Server Error
POST	/api/AMS/coaches	✔ Yes	Create a new coach	Object (required): sport_id (UUID string), name (string)	None	201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error
PATCH	/api/AMS/coaches/{id}	✔ Yes	Update coach details	Object (partial): sport_id (UUID string, optional), name (string, optional)	id (string, uuid, path)	200 OK, 400 Bad Request, 404 Not Found, 409 Conflict, 500 Internal Server Error
DELETE	/api/AMS/coaches	✔ Yes (should)	Bulk delete	Object: ids (array of	None	200 OK, 400 Bad

		be Admin per docs)	coaches	UUID strings, min 1)		Request, 409 Conflict, 500 Internal Server Error
GET	/api/AMS/coaches/mappings	✗ No	List all coach-athlete mappings	None	None	200 OK, 500 Internal Server Error
GET	/api/AMS/coaches/mappings/athlete/{athleteId}	✗ No	List mappings for a specific athlete	None	athleteId (string, uuid, path)	200 OK, 400 Bad Request, 500 Internal Server Error
POST	/api/AMS/coaches/mappings	✓ Yes	Create coach-athlete mapping	Object (required): athlete_id (UUID string), coach_id (UUID string), is_active (boolean, optional, default true)	None	201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error
PATCH	/api/AMS/coaches/mappings/{athleteId}/{coachId}	✓ Yes	Update mapping status	Object (required): is_active (boolean)	athleteId (string, uuid, path), coachId (string, uuid, path)	200 OK, 400 Bad Request, 404 Not Found, 500 Internal Server Error
DELETE	/api/AMS/coaches/mappings	✓ Yes	Bulk delete mappings	Array<Object> (min 1): [{ athlete_id (UUID), coach_id (UUID) }]	None	200 OK, 400 Bad Request, 500 Internal Server Error

Nutritionists

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/AMS/nutritionists/my-athletes	✓ Yes	Get assigned	None	nutritionist_id (UUID,	200 OK, 400 Bad Request,

			athletes (sorted by pinned)		query, optional)	401 Unauthoriz ed, 500 Internal Server Error
PATCH	/api/AMS/nutritionists/pin	✔ Yes	Pin / unpin athlete	Object (required): athlete_id (UUID), is_pinned (boolean), nutritionist_i d (UUID, optional)	None	200 OK, 400 Bad Request, 404 Not Found, 500 Internal Server Error
GET	/api/AMS/nutritionists	✗ No	List all nutritioni sts	None	None	200 OK, 500 Internal Server Error
POST	/api/AMS/nutritionists	✔ Yes	Create nutritioni st	Object (required): name (string)	None	201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error
DELETE	/api/AMS/nutritionists	✔ Yes	Bulk delete nutritioni sts	Object: ids (array of UUID strings, min 1)	None	200 OK, 400 Bad Request, 500 Internal Server Error
GET	/api/AMS/nutritionists/mappings	✗ No	List all nutritioni st- athlete mapping s	None	is_active (boolean, query, optional)	200 OK, 400 Bad Request, 500 Internal Server Error
GET	/api/AMS/nutritionists/mappings/athlete/{athleteId}	✗ No	List mapping s by athlete	None	athleteId (UUID, path), is_active (boolean, query, optional)	200 OK, 400 Bad Request, 500 Internal Server Error
POST	/api/AMS/nutritionists/mappings	✔ Yes	Create nutritioni st- athlete mapping	Object (required): athlete_id (UUID), nutritionist_i	None	201 Created, 400 Bad Request, 409

				d (UUID), is_active (boolean, optional, default true)		Conflict, 500 Internal Server Error
DELETE	/api/AMS/nutritionists/mappings	✓ Yes	Bulk delete mappings	Array<Object> (min 1): [{ athlete_id (UUID), nutritionist_id (UUID) }]	None	200 OK, 400 Bad Request, 500 Internal Server Error

Sports

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/AMS/sports	✗ No	List all sports (default: active only, optional include inactive)	None	includeInactive (boolean, query, optional, default: false)	200 OK, 500 Internal Server Error
POST	/api/AMS/sports	✓ Yes	Create a new sport	Object (required): sport (string)	None	201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error
DELETE	/api/AMS/sports	✓ Yes (should be Admin per docs)	Bulk delete sports	Object: ids (array<UUID string>, min 1)	None	200 OK, 400 Bad Request, 409 Conflict, 500 Internal Server Error
PATCH	/api/AMS/sports/{id}	✓ Yes (should be Admin per docs)	Update sport active status	Object (required): is_active (boolean)	id (string, uuid, path)	200 OK, 400 Bad Request, 404 Not Found, 500 Internal Server Error

7.3 Auth Endpoints

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
POST	/api/auth/register	✗ No (rate limited)	Register a new user	Object (required): email (string, email), password (string, min 8), first_name (string),	None	201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error

				last_name (string), role (enum: IT_ADMIN, ADMIN, NUTRITIONIST, COACH, ATHLETE, optional, default NUTRITIONIST)		
POST	/api/auth/login	✗ No (rate limited)	Login (Step 1: send verification code)	Object (required): email (string, email), password (string)	None	200 OK, 401 Unauthorized, 403 Forbidden, 500 Internal Server Error
POST	/api/auth/verify-code	✗ No (rate limited)	Login (Step 2: verify code, return JWT)	Object (required): email (string, email), code (string, 6-digit)	None	200 OK, 401 Unauthorized, 500 Internal Server Error
POST	/api/auth/resend-code	✗ No (rate limited)	Resend verification code	Object (required): email (string, email)	None	200 OK, 404 Not Found, 500 Internal Server Error
GET	/api/auth/me	✓ Yes	Get current authenticated user	None	None	200 OK, 401 Unauthorized, 403 Forbidden
POST	/api/auth/logout	✓ Yes	Logout user	None	None	200 OK

7.4 Admin Endpoints

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/admin/users	Yes	Get all users with optional filters	—	Query: role, is_active, search	200: list of users
GET	/api/admin/users/{id}	Yes	Get user details	—	Path: id (uuid)	200: user details, 404: not found
PATCH	/api/admin/users/{id}/active	Yes	Activate or deactivate a user	{ is_active: boolean }	Path: id (uuid)	200: updated user, 400: self-modify blocked, 404

PATCH	/api/admin/users/{id}/password	Yes	Change user password (invalidates sessions)	{ new_password: string, confirm_password: string }	Path: id (uuid)	200 success, 400 validation error, 404
PATCH	/api/admin/users/{id}/email	Yes	Change user email (2FA email)	{ new_email: string, reset_verification?: boolean }	Path: id (uuid)	200 success, 400 self-change blocked, 409 email exists
PATCH	/api/admin/users/{id}	Yes	Update user details (role, name, status)	{ first_name?, last_name?, role?, is_active?, is_email_verified? }	Path: id (uuid)	200 success, 400 invalid update, 404
DELETE	/api/admin/users/{id}	Yes	Delete user and all related data	—	Path: id (uuid)	200 success, 400 self-delete blocked, 404
GET	/api/admin/users/{id}/activity	Yes	Get user activity logs	—	Path: id (uuid)	200 activity data, 404
GET	/api/admin/audit-logs	Yes	Get audit logs with filters	—	Query: user_id, table_name, action, start_date, end_date, limit	200 audit logs
GET	/api/admin/audit-logs/statistics	Yes	Get audit log statistics	—	Query: user_id?	200 statistics
GET	/api/admin/audit-logs/tables	Yes	Get list of audited tables	—	—	200 table list
GET	/api/admin/audit-logs/{tableName}/{recordId}	Yes	Get audit history for a record	—	Path: tableName, recordId; Query: limit?	200 record history

7.5 OCR/Python Endpoints

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
POST	/api/ocr/ocr-only	No	Extract raw text from image (Stage 1 OCR for editable flow)	multipart/form-data → file (image)	None	200: raw OCR text + metadata 400: invalid file 422: OCR failure 503: service unavailable 504: timeout
POST	/api/ocr/analyze-text	No	Structure raw OCR text + optionally generate embeddings	JSON → raw_text (string), generate_vectors (boolean)	None	200: structured text + vectors 400: invalid body 422: analysis failure 503: service unavailable 504: timeout
POST	/api/ocr/find-alternatives	No	Find similar supplements using verified nutrition data (vector search ≥60%)	JSON → ingredients[], nutritional_per_serving[], nutritional_per_100g[], serving_size_grams?, page?, per_page?	None	200: similar supplements list 422: insufficient data 503: service unavailable 504: timeout
POST	/api/ocr/analyze	No	Full pipeline: OCR + structuring + similarity search	multipart/form-data → file (image)	Query: page, per_page	200: extracted data + similar supplements 400: invalid input 503: service unavailable 504: timeout
POST	/api/ocr/extract	No	Extract brand, product name, and batch ID from images	multipart/form-data → brand_image (required), batch_image (optional)	None	200: extracted brand/product/batch info 400: missing brand image 503: service unavailable
POST	/api/ocr/verify	No	Verify supplement batch/product across 6 certification databases	JSON → supplement_brand, supplement_name, batch_id?	None	200: verification results + sources 400: invalid request 503: service unavailable 504: timeout (extended 360s)
POST	/api/ocr/upload	No (legacy)	Legacy OCR endpoint (basic text extraction only)	multipart/form-data → file	None	200: OCR result 400: invalid file 500: OCR failure

7.6 Consultation Endpoints

Actionables (Open Items)

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/actionables/statuses	No	Get active open item statuses	None	None	200 list, 500 error
GET	/api/Consultation/actionables/session/{sessionId}	No	Get open items for session (sorted by due date)	None	sessionId (path, uuid)	200 list, 400, 404, 500
POST	/api/Consultation/actionables	Yes	Create open item	sessions_id, open_item_status_id (+ optional fields)	None	201 created, 400, 500
PATCH	/api/Consultation/actionables/{id}	Yes	Update open item (partial update)	Partial fields (status, description, etc.)	id (uuid)	200 updated, 400, 404, 500
DELETE	/api/Consultation/actionables	Yes	Bulk delete open items	ids: string[]	None	200 deleted, 400, 500

Anthropometry

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/sessions/{sessionId}/anthropometry	Yes	Get anthropometry + computed fields	None	sessionId (uuid)	200 data, 400, 404, 500
PATCH	/api/Consultation/sessions/{sessionId}/anthropometry	Yes	Update anthropometry (partial)	Partial anthropometry fields	sessionId (uuid)	200 updated, 400, 404, 500

Lookups

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/lookups/consult-types	No	Get active consult types	None	None	200 list, 500
GET	/api/Consultation/lookups/consultation-objectives	No	Get active consultation objectives	None	None	200 list, 500

Session

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/consultation-session/upcoming	No	Get upcoming sessions	None	None	200
GET	/api/Consultation/consultation-session/today	Yes	Get today's sessions for nutritionist	None	None	200
GET	/api/Consultation/consultation-session/range	No	Get sessions by date range	None	query params (range)	200
GET	/api/Consultation/consultation-session/athlete/{athleteId}/latest	No	Get latest session for athlete	None	athleteId	200, 404
GET	/api/Consultation/consultation-session/athlete/{athleteId}/all	No	Get all sessions for athlete	None	athleteId	200
GET	/api/Consultation/consultation-session/{id}	No	Get session details	None	id	200
POST	/api/Consultation/consultation-session	Yes	Create session	athlete_id, type_of_consult_id (+ optional fields)	None	201 created
PATCH	/api/Consultation/consultation-session/{id}	Yes	Update session	Partial session fields	id	200 updated

PATCH	/api/Consultation/consultation-session/{id}/status	Yes	Update session status	status field	id	200
DELETE	/api/Consultation/consultation-session/{id}	Yes	Cancel session	None	id	200

Meal Log

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/sessions/{sessionId}/meal-log	Yes	Get meal log + sleep data	None	sessionId	200
PUT	/api/Consultation/sessions/{sessionId}/meal-log	Yes	Upsert meal log + sleep	entries, mealOtherRemarks, sleep	sessionId	200

Medical History

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/medical-history/athlete/{athleteId}/general	No	Get athlete general medical record	None	athleteId	200
GET	/api/Consultation/medical-history/athlete/{athleteId}/eligibility	No	Get DOB, gender, age	None	athleteId	200
GET	/api/Consultation/medical-history/{sessionId}	No	Get full medical history card	None	sessionId	200
POST	/api/Consultation/medical-history	Yes	Create / upsert medical history	sessions_id + optional sections	None	201
PATCH	/api/Consultation/medical-history/{sessionId}	Yes	Update medical history (partial)	Partial medical history fields	sessionId	200

Nutrition Diagnosis Summary

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/nutrition	No	Get nutrition details for a specific	—	Path: sessionId (UUID)	200: nutrition diagnosis data, 400:

	ion-diagnosis-summary/{sessionId}		session. Returns session note fields or null if not found.			bad request, 404: not found 500: server error
POST	/api/Consultation/nutrition-diagnosis-summary	Yes	Create or update (upsert) nutrition diagnosis for a session.	sessions_id (UUID, required)main_nutrition_diagnosiscarbohydrates_reviewprotein_reviewfat_reviewother_reviewfollow_up_noteintervention_noteother_remarks	—	201: created/saved successfully 400: bad request 404: not found 500: server error
PATCH	/api/Consultation/nutrition-diagnosis-summary/{sessionId}	Yes	Update nutrition diagnosis fields (partial update, upsert supported).	main_nutrition_diagnosiscarbohydrates_reviewprotein_reviewfat_reviewother_reviewfollow_up_noteintervention_noteother_remarks	Path: sessionId (UUID)	200: updated successfully 400: bad request 404: not found 500: server error

Nutrition Requirements

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/sessions/{sessionId}/nutrition-requirements	Yes	Retrieve nutrition requirements (inputs + computed outputs) for a session.	—	Path: sessionId (UUID)	200: nutrition requirements data 400: bad request 404: not found 500: server error
PATCH	/api/Consultation/sessions/{sessionId}/nutrition-requirements	Yes	Update nutrition requirement inputs for a session (partial update).	NutritionRequirementsPatchRequest (schema-based payload; partial fields allowed)	Path: sessionId (UUID)	200: updated nutrition requirements data 400: bad request 404: not found 500: server error

Supplement Dispensing

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
GET	/api/Consultation/supplement-	No	Get all prescriptions linked	—	Path: sessionId (UUID)	200: list of prescriptions, 400:

	dispensing/session/{sessionId}		to a session, including batch and supplement details (auto-populated).			bad request, 404: not found, 500: server error
POST	/api/Consultation/supplement-dispensing	Yes	Create a new prescription linked to a session and inventory batch, with stock validation.	sessions_id (UUID, required), batch_id (UUID, required), prescribed_quantity (integer, required), dosage, dosage_unit, dosage_frequency, start_date, projected_end_date, follow_up_required, other_remarks	—	201: created successfully, 400: bad request, 409: insufficient stock, 500: server error
PATCH	/api/Consultation/supplement-dispensing/{id}	Yes	Partially update an existing prescription (only provided fields are updated).	batch_id, prescribed_quantity, dosage, dosage_unit, dosage_frequency, start_date, projected_end_date, follow_up_required, other_remarks	Path: id (UUID)	200: updated successfully, 400: bad request, 404: not found, 409: insufficient stock, 500: server error
DELETE	/api/Consultation/supplement-dispensing/{id}	Yes	Permanently delete a prescription from a session.	—	Path: id (UUID)	200: deleted successfully, 400: bad request, 404: not found, 500: server error

Training Schedule

Method	Endpoint	Auth Required	Summary	Request Body	Parameters	Responses
--------	----------	---------------	---------	--------------	------------	-----------

GET	/api/Consultation/sessions/{sessionId}/training-schedule	Yes	Retrieve the training schedule for a specific session.	—	Path: sessionId (UUID)	200: training schedule data, 400: bad request, 500: server error
PUT	/api/Consultation/sessions/{sessionId}/training-schedule	Yes	Create or update (upsert) the training schedule for a session.	TrainingSchedulePayload (schema-based object)	Path: sessionId (UUID)	200: upserted training schedule, 400: bad request, 500: server error

8. Security Architecture

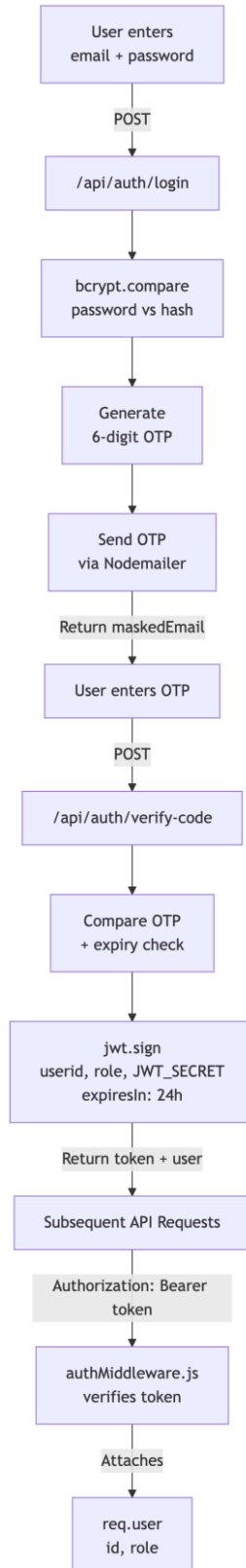
8.1 Implemented Controls

Control	Implementation
Password storage	bcrypt with salt rounds
Authentication	Email + OTP two-factor with 10-minute expiry
Session tokens	JWT signed with HS256, 24h expiry, stored client-side
Rate limiting	express-rate-limit on all auth endpoints
SQL injection prevention	Parameterised queries throughout (no string interpolation)
Input validation	Zod schemas on all API inputs
UUID validation	Validated before DB queries to prevent malformed lookups
Environment secrets	All credentials in .env files, never committed
CORS	Configured to allow only the frontend origin

8.2 Planned Controls

Control	Status
Role-based access control (RBAC) enforcement	Roles stored but not enforced at route level yet
Helmet.js security headers	Planned
CSRF protection	Planned
JWT refresh tokens	Planned
Token blacklisting on logout	Planned

8.3 Authentication Flow



9. Setup & Deployment System setup and environment

NutriFusion is deployed as a three-service stack — a Next.js 15 frontend (port 3000), an Express 5 backend (port 8000), and a FastAPI Python microservice (port 8001) — orchestrated by Docker Compose. This section describes how to bring the full system up from scratch using the Docker-based setup, which is the recommended approach for both development and production environments.

Prerequisites

Ensure the following are available before proceeding:

- **Docker Desktop 4.x or later**, which includes Docker Engine and Docker Compose v2. Docker Desktop for Mac and Windows can be downloaded from the Docker website. On Linux, install Docker Engine and the Compose plugin separately.
- **A PostgreSQL 14+ database with the pgvector extension enabled**. The default deployment targets an externally hosted database (such as AWS RDS). A fully self-contained local database option is also available — refer to the Optional: Local Database subsection below.
- **An OpenAI API key with sufficient token balance**, used by the Python microservice for GPT-4o-mini structured OCR text parsing and LLM-based product data enrichment during web scraping.
- **A Gmail account with a Google App Password configured**. The backend uses this account to dispatch six-digit 2FA login codes to users during authentication.

Environment Configuration

All configurations are supplied through a single root-level `.env` file. `docker-compose.yml` reads from this file automatically on startup and injects the relevant variables into each container's environment. A fully annotated template is provided at `.env.example` in the repository root.

Create the environment file by running:

```
cp .env.example .env
```

Open the generated `.env` and populate the values described in the table below. Variables marked as required have no safe default and must be supplied.

Variable	Description	Required	Default
PGHOST	Hostname of the PostgreSQL database server	Yes	—
PGPORT	PostgreSQL port	No	5432
PGDATABASE	Name of the target database	Yes	—
PGUSER	Database user	Yes	—
PGPASSWORD	Database password	Yes	—

PGSSLMODE	SSL mode. Set to “require” for AWS RDS; set to “disable” for the local containerised database.	No	require
JWT_SECRET	Secret key used to sign and verify JWT session tokens	Yes	—
JWT_EXPIRY	Token expiry duration (e.g. 24h, 7d)	No	24h
EMAIL_USER	Gmail address used to dispatch 2FA codes	Yes	—
EMAIL_PASSWORD	Google App Password for the Gmail account	Yes	—
SKIP_2FA	Set to true to bypass 2FA during local development	No	false
CODE_EXPIRY_MINUTES	Minutes before a 2FA code expires	No	10
MAX_VERIFICATION_ATTEMPTS	Maximum 2FA attempts before IP lockout	No	3
OPENAI_API_KEY	OpenAI API key for OCR parsing and web scraper enrichment	Yes	—
EMBEDDING_MODEL	Sentence transformer model for supplement vectorisation	No	BAAI/bge-small-en-v1.5
VECTOR_DIMENSION	Dimensionality of generated embedding vectors	No	384

Starting the System

With the .env file populated, build the Docker images and start all services with a single command executed from the repository root:

```
docker compose up --build
```

On the first run, Docker builds three images from the source. The Python image is the most time-intensive: it downloads and compiles PyTorch, PaddleOCR, Playwright, and their system dependencies inside the container. Expect an initial build time of 20–40 minutes. All build layers are cached after the first run; subsequent cold starts complete in under a minute.

Services are started in dependency order, gated by container health checks:

- Python Services starts first. Its health check polls GET /health every 30 seconds (with a 60-second start-up period) before the service is marked healthy.
- Backend starts after Python is healthy. Its health check polls GET /api/test.
- Frontend starts last, once the backend reports healthy.

To run the stack in the background (detached mode):

```
docker compose up --build -d
```

Verifying the Installation

Once all three containers report a healthy status (visible via `docker compose ps` or the Docker Desktop dashboard), verify each service by opening the following URLs:

Service	URL	Expected Response
Frontend	http://localhost:3000	Login page
Backend Swagger UI	http://localhost:8000/docs	Interactive API documentation
Backend health check	http://localhost:8000/api/health	<code>{"status":"healthy"}</code>
Python Services docs	http://localhost:8001/docs	FastAPI auto-generated documentation
Python health check	http://localhost:8001/health	<code>{"status":"healthy"}</code>

Smoke Test

Navigate to <http://localhost:3000> and log in with a valid user account. The system dispatches a six-digit 2FA code to the registered email address. Successfully entering the code and landing on the dashboard confirms that the database connection, backend authentication flow, 2FA email dispatch, frontend-to-backend API proxy, and Python microservice health check are all functioning correctly.

To skip 2FA during local development, set `SKIP_2FA=true` in `.env` and rebuild the backend:

```
docker compose up --build backend
```

Common Management Commands

Command	Description
<code>docker compose up --build</code>	Build images and start all services
<code>docker compose up --build -d</code>	Build and start in detached (background) mode
<code>docker compose down</code>	Stop and remove all containers
<code>docker compose ps</code>	List running containers and their status
<code>docker compose logs -f</code>	Stream live logs from all services
<code>docker compose logs -f backend</code>	Stream live logs from the backend service only
<code>docker compose logs -f python</code>	Stream live logs from the Python microservice only
<code>docker compose up --build backend</code>	Rebuild and restart a single service after a code change

Optional: Local Database

By default, `docker-compose.yml` connects to an externally-hosted PostgreSQL instance specified by `PGHOST`. For a fully self-contained local setup — useful when no external database is

available — open `docker-compose.yml` and uncomment the `db:` service block and its associated volumes: declaration at the bottom of the file. Then set `PGSSLMODE=disable` in `.env`, as the local container does not use SSL.

The containerised database starts with an empty schema. The NutriFusion database schema and seed data must be applied to the running container before the application functions correctly.

Troubleshooting

Symptom	Likely Cause and Resolution
Frontend shows a network error on login	The backend container is not yet healthy. Check <code>docker compose ps</code> and wait for all containers to report healthy, then refresh the page.
Python service container exits on startup	A required environment variable (<code>OPENAI_API_KEY</code> , <code>PGHOST</code> , etc.) is missing or incorrect. Run <code>docker compose logs python</code> to see the specific error.
Database connection refused	<code>PGHOST</code> , <code>PGUSER</code> , or <code>PGPASSWORD</code> is incorrect, or <code>PGSSLMODE</code> does not match the database configuration (use <code>disable</code> for a local container, <code>require</code> for AWS RDS).
Port already in use	Another process is occupying port 3000, 8000, or 8001. Stop the conflicting process before running <code>docker compose up</code> .
First build stalls at PaddleOCR or PyTorch	Expected behaviour — the Python image installs large ML dependencies and takes 20–40 minutes on the initial build. Do not interrupt the build.

Email Setup

To send two-factor authentication emails, the project uses `nodemailer`. For security reasons, an app-specific password should be used and stored in the `env` file under `EMAIL_PASSWORD` instead of a standard account password. The steps to set up an app-specific password for a Gmail account are as follows:

1. Go to <https://myaccount.google.com/security>
2. Enable 2-Step Verification
3. Go to <https://myaccount.google.com/apppasswords>
4. Create app password for "Mail"
5. Use that password in `EMAIL_PASSWORD`
6. Other email account types should have their own methods for creating app-specific passwords.

10. Design Decisions & Patterns

10.1 Why Separate Python Service?

Three reasons drove the decision to run AI/ML as a separate FastAPI microservice:

1. **Vector consistency** — All supplement embeddings must be generated by the same model instance. Centralising generation in Python prevents model drift that would occur if Node.js and Python produced incompatible vectors.
2. **Python library ecosystem** — PaddleOCR, PyTorch, Playwright, and sentence-transformers are Python-first. Their Node.js bindings are either absent or significantly less mature.
3. **Resource isolation** — OCR and vectorisation are CPU-intensive operations. Running them in a separate process prevents blocking the Node.js event loop during inference.

10.2 Why PostgreSQL with pgvector over a Dedicated Vector DB?

Supplement similarity search operates on a relatively small dataset (hundreds to low thousands of records). Using pgvector within the existing PostgreSQL instance avoids introducing a second database technology (e.g. Pinecone, Weaviate) while still supporting efficient cosine similarity queries. The `<=>` operator and `vector(384)` columns integrate naturally with the existing SQL queries.

10.3 Why Parameterised Queries over an ORM?

The team chose raw parameterised SQL over an ORM (e.g. Prisma, TypeORM) to maintain full control over query structure, enable complex JOINS across multiple schemas, and avoid ORM abstraction overhead. This is appropriate for a capstone project where learning SQL is a stated goal.

10.4 Why Zod over Joi?

Zod's TypeScript-first design provides better type inference and a more modern API. It also prepares the backend for a potential future TypeScript migration, where Zod schemas can be shared between frontend and backend.

10.5 Common Code Patterns

Dynamic PATCH (field mapping)

Controllers build update queries by iterating over a field map and appending only the fields present in the request body. This prevents accidental nullification of unrelated columns.

Bulk Delete

All delete endpoints accept `{ ids: ["uuid1", "uuid2"] }` in the request body, validated against a reusable `bulkDeleteSchema`.

Fire-and-forget Scraping

`POST /api/SSS/scraping/start` immediately returns `202 Accepted` and submits scrape requests to the Python service without awaiting their completion. Results are later found in the staging table.

Transaction Pattern for Multi-table Creates

```
const client = await pool.connect();
try {
  await client.query('BEGIN');
  // ... multiple inserts ...
  await client.query('COMMIT');
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();
}
```